

Robust Object Classification for Robotics Using Classical Descriptors, CNNs, and Transfer Learning

Iva Jorgusheska, UID:11114620

1. Introduction

Robotics aims to build autonomous systems that perceive, reason, and act in the physical world, with vision playing a central role in manipulation, navigation, localisation, and human-robot interaction [5]. Reliable visual perception directly affects robot performance, generalisation, and safety [22]. Classical robotic vision methods based on hand-crafted features such as SIFT [9], SURF [3], HOG [2], and Bag-of-Visual-Words [4] were effective in controlled settings but brittle under occlusion, illumination change, clutter, and noise [19]. Their limitations prompted the shift toward learned representations, driven by deep CNNs following ImageNet breakthroughs [8], now widely used in detection, segmentation, pose estimation, and grasping [13]. Recent advances extend to transformers [29]; [28] and vision-language models enabling instruction following and semantic reasoning [38], alongside a broader move from geometric pipelines to deep learning across manipulation, navigation, SLAM, and HRI [30]. Yet challenges remain, including data demands, computational cost, real-time constraints, safety, and sim-to-real transfer [39] [32]. This report compares classical local-feature-based and deep-learning-based object recognition across benchmark datasets, using systematic hyperparameter exploration to analyse their strengths and limitations within contemporary robotic vision.

2. State of the Art in Computer Vision for Robotics

Computer vision (CV) is one of the core components of robotic intelligence, enabling robots to interpret their environment, recognise objects, understand scenes, interact with humans, and make decisions based on visual input [5]. Modern surveys consistently show that perception is a prerequisite for manipulation, navigation, SLAM, and autonomous control [30]. Over the past decade, the field has shifted from hand-engineered features to deep-learning-based perception, enabling far more robust performance in real-world robotic applications.

2.1. Classical Computer Vision in Robotics

Before deep learning, robotic vision relied on hand-crafted local features and geometric pipelines. Algorithms such as **SIFT** extracted scale- and rotation-invariant keypoints for object recognition and visual SLAM [9], while **SURF** accelerated this approach using integral images and a fast Hessian detector suitable for embedded real-time systems [3]. For tasks such as pedestrian detection and mobile navigation, **HOG** provided dense gradient descriptors widely used in classical safety and perception systems [2].

Built on these descriptors, **Bag-of-Visual-Words (BoVW)** models clustered features into visual vocabularies and used histograms with linear SVMs for category-level recognition with limited labelled data. In robotics, these representations were combined with geometric estimation (RANSAC, PnP, bundle adjustment) and formed the basis of feature-based SLAM pipelines, as **ORB-SLAM**, where robust feature matching is essential for tracking, mapping, and loop closure [4].

Classical pipelines offered advantages: interpretability, low data requirements, well-understood failure modes, and real-time feasibility, but struggled with clutter, illumination changes, textureless surfaces, occlusion, and domain shift [27]. They also required extensive manual engineering for each task.

In this coursework, my **local-feature baseline** follows this classical paradigm: extracting engineered descriptors, constructing BoVW-like representations, and training a shallow classifier. This mirrors the historical foundations of robotic perception and provides a meaningful comparison point for evaluating how a modern CNN improves robustness and generalisation.

2.2. CNNs: the Deep Learning Turning Point

A major breakthrough in robotic perception came with **convolutional neural networks (CNNs)** trained end-to-end on large image datasets. *AlexNet* showed that deep CNNs on ImageNet could outper-

form hand-crafted pipelines by a large margin, using stacked convolutions, ReLU activations, pooling, and fully connected layers [16]. Later architectures such as *VGG* (very deep networks with small 3×3 kernels) and *ResNet* (residual connections for stable deep training) further improved accuracy and generalisation [11] [15].

CNNs include inductive biases that make them particularly effective for robotic vision:

- **Local receptive fields and weight sharing**, which efficiently capture edges, corners, and textures in high-resolution imagery [8].
- **Approximate translation equivariance**, ensuring stable feature responses as objects move within the image, beneficial for mobile robots and manipulators [32].

These properties make CNNs far more robust than classical descriptors under clutter, dynamics, or poor lighting. Consequently, CNN backbones such as VGG and ResNet have become standard feature extractors for **object detection**, **semantic segmentation**, **grasp prediction**, and **scene understanding**, replacing SIFT/BoVW pipelines in most robotics applications [27]. Surveys highlight this transition from manually designed features to learned representations that can be fine-tuned for detection, pose estimation, and semantic mapping with far less task-specific engineering [13].

In this coursework, my **CNN classifiers** reflects this paradigm shift: rather than constructing features explicitly, the network learns hierarchical visual representations directly from data, mirroring the broader move in robotic perception from classical CV pipelines to deep learning.

2.3. Transformers and Vision Transformers (ViT)

Transformers, originally developed for language modelling, have recently been adapted to computer vision. The **Vision Transformer (ViT)** replaces convolutions with patch-based self-attention: images are split into fixed-size patches, linearly embedded with positional encodings, and processed by a Transformer encoder, with a class token summarising the entire image [29].

Unlike CNNs, ViTs include minimal inductive bias regarding locality or translation, making them more data-hungry but highly flexible when pre-trained on large image corpora. While CNNs dominate on smaller datasets, ViTs can surpass ResNet-style architectures at scale and transfer effectively to robotic perception tasks such as detection, segmentation, and navigation [29].

Transformers have also been applied to object detection. **DETR** formulates detection as a set prediction problem, combining a CNN backbone with a Transformer encoder-decoder and outputting a fixed number of object queries, removing anchor boxes and non-maximum suppression [31].

Recent surveys report increasing use of transformer architectures in robotics, including manipulation, autonomous navigation, gesture recognition, and multi-view perception, where global self-attention helps capture long-range spatial relationships important for planning in cluttered environments [36].

2.3.1. 3D Perception and Point-Cloud Networks

Modern robotic systems increasingly rely on 3D perception from depth cameras and LiDAR, where architectures such as **PointNet** and **PointNet++** operate directly on unordered point clouds using shared MLPs and hierarchical grouping to capture both global and local structure [20]. Surveys in visual SLAM now treat deep networks for depth prediction, semantic segmentation, and learned 3D features as standard components of mapping pipelines in complex indoor and outdoor environments [34].

2.3.2. Vision-Language Models and Semantic Generalisation

Recent advances in **vision-language models (VLMs)**, such as **CLIP**, jointly learn from images and text using contrastive objectives, enabling robots to perform open-vocabulary recognition and zero-shot

classification through language prompts [33]. Surveys show that VLMs provide richer semantic understanding and stronger generalisation to unseen categories than purely visual CNNs or ViTs, making them increasingly attractive as perception backbones for manipulation, navigation, and instruction following. Foundation-model research further argues that large pre-trained VLMs can reduce robot-specific data requirements and support more flexible, language-driven robotic behaviours [40].

2.4. Computer Vision Tasks in Robotics

- **Object recognition:** Robots identify and categorise objects in their workspace using traditional features such as SIFT [9], SURF [3], HOG [2], while deep CNNs enable robust detection in cluttered environments and are widely used in grasping and warehouse automation [27].
- **Scene understanding:** Detection and segmentation models such as FCN [14] and Mask R-CNN [18] support collision avoidance and planning, with transformers improving large-scale scene perception through global attention.
- **SLAM:** Classical systems like ORB-SLAM rely on geometric feature matching, while deep learning enhances depth estimation, loop closure, and localisation in modern SLAM pipelines [34].
- **Vision-based navigation:** Deep networks enable end-to-end navigation and imitation learning, outperforming earlier hand-crafted or rule-based approaches, with CNNs and transformers increasingly used for aerial and underwater robots [35].
- **Manipulation and grasping:** Deep grasp detection directly predicts grasp points from images, outperforming classical geometric methods, while large-scale visuomotor learning enables robots to acquire manipulation skills from raw pixels [23].
- **Human-robot interaction (HRI):** Deep learning dominates gesture, pose, and intention recognition, which are essential for safe collaboration with humans in industrial and healthcare settings [17].
- **Vision-based feedback and control:** While classical visual servoing relied on geometric models, deep neural policies now map images directly to motor commands, improving performance in complex control tasks [1].

2.5. Critical Analysis: Strengths and Limitations of Deep Learning in Robotics

Deep learning has significantly advanced robotic vision by improving robustness to clutter, occlusion, viewpoint changes, and sensor noise, outperforming classical pipelines in detection, segmentation, pose estimation, and visuomotor control [27]. However, recent surveys highlight several persistent challenges:

- **Data requirements:** Deep models require large, task-specific datasets, but collecting labelled robotic data is costly and often unsafe [39].
- **Generalisation and domain shift:** Models trained in one environment may fail under new lighting, textures, or layouts, requiring domain adaptation or continual learning [29].
- **Sim-to-real gap:** Policies trained in simulation still suffer from mismatches in physics, sensing, and dynamics when deployed on real robots [32].
- **Time constraints:** State-of-the-art CNNs, transformers, and VLMs often exceed the compute and latency budgets of mobile robots, requiring compression or specialised hardware [1].
- **Safety and reliability:** Deep models are vulnerable to out-of-distribution inputs, adversarial perturbations, and partial observability, raising safety concerns in human-robot environments [23].
- **Lack of interpretability:** Unlike classical features, deep models are opaque and difficult to verify for safety-critical tasks [37].
- **Bias and ethics:** Vision-language models inherit biases from web-scale data, which can influence robot perception and decision-making [37].
- **Rare or unsafe scenarios:** Fragile objects, transparent surfaces, and hazardous events are underrepresented in datasets, limiting

robustness in safety-critical conditions [37].

These limitations contrast with classical local-feature methods, which require little data, are interpretable, and run efficiently on low-power hardware but struggle in cluttered or poorly lit environments. The two approaches evaluated in this coursework: a classical local-feature baseline and a CNN classifier reflect this broader trade-off: classical CV offers transparency and data efficiency, while deep learning provides greater robustness and generalisation at the cost of higher data and compute demands. This mirrors the current landscape of robotic vision, where hybrid pipelines are increasingly used to balance performance, safety, and resource constraints.

3. Datasets Description

I use three visually distinct datasets, **CIFAR-100**, **RGB-D Object**, and **iCubWorld**, chosen for their differences in resolution, variability, and realism. Together they allow a controlled comparison of classical local features and deep learning models.

3.1. CIFAR-100

CIFAR-100 contains 60,000 RGB images at **32×32** resolution, offering very low spatial detail. This limits SIFT/SURF, which extract only a few keypoints per image, while CNNs can still leverage coarse colour and shape cues. Its combination of low resolution, high class diversity, and cluttered backgrounds makes it a challenging benchmark and a useful contrast between classical and learned vision methods [6].

3.2. RGB-D Object Dataset

The RGB-D dataset provides **640×480** RGB images with depth maps captured in controlled turntable sequences [7]. For this project, eleven well-separated object classes were selected, making it the **easiest** dataset: clean backgrounds, consistent viewpoints, and distinctive geometry give SIFT/SURF hundreds of stable keypoints and create favourable conditions for CNNs. Although only RGB was used here, the dataset is widely used in robotics for object recognition and pose estimation.

3.3. iCubWorld Dataset

iCubWorld consists of images acquired directly by the **iCub humanoid robot** [10], at **320×240** or **640×480** resolution. Unlike RGB-D, it reflects real robotic sensing: variable lighting, background clutter, hand and arm occlusions, and motion blur from robot movement. These conditions make keypoint detection unstable and increase appearance variability, making iCubWorld the most realistic and challenging dataset for evaluating both classical and deep learning pipelines.

4. Classical Computer Vision Pipeline

For the classical baseline, I implemented a Bag-of-Visual-Words (BoW) pipeline using dense SIFT or dense ORB descriptors, a K-Means visual vocabulary, and a linear SVM classifier. This reproduces a standard pre-deep-learning recognition method and provides a controlled comparison against the CNN model. All images were resized to 128×128 to ensure consistent descriptor density across CIFAR-100, RGB-D Objects, and iCubWorld. Each dataset was split into 70% training, 15% validation, and 15% testing, with hyperparameters selected using the validation split.

4.1. Dense Local Feature Extraction (SIFT and ORB)

Classical pipelines describe images using local patch descriptors rather than learning features end-to-end. Because traditional keypoint detectors fail on low-resolution or low-texture images, I used dense sampling to ensure a uniform number of descriptors per image.

4.1.1. Dense SIFT

SIFT produces 128-dimensional gradient-based descriptors that are stable under scale, rotation, and moderate lighting changes. In this project, I applied SIFT on a regular grid, where the spacing and patch size determine descriptor density and receptive field.

Parameters tested:

- $\text{step} \in \{8, 12\}$
- $\text{patch_size} \in \{8, 16\}$

Smaller steps and patches provide fine detail suitable for small images such as CIFAR-100, while larger values capture broader texture patterns in higher-resolution datasets like RGB-D and iCubWorld.

SIFT's continuous, high-dimensional descriptors generally provide strong discriminative power for category-level recognition.

4.1.2. Dense ORB

ORB is a computationally efficient binary descriptor designed for real-time vision tasks. It combines FAST-based corner detection with BRIEF binary tests and orientation compensation. As with SIFT, ORB was extracted densely to allow direct comparison across datasets.

Parameters used:

- number of features = 5000
- $\text{step} \in \{4, 8\}$
- $\text{patch_size} \in \{8, 16\}$

ORB's low computational cost allows denser sampling than SIFT. Although binary descriptors are less expressive than gradient-based ones, they remain widely used in robotics due to their speed and suitability for embedded hardware. Including ORB therefore provides insight into how lightweight descriptors behave on visually diverse datasets.

4.2. Building the Visual Vocabulary (K-Means)

After extracting descriptors, I created a visual vocabulary using Mini-Batch K-Means. The vocabulary groups similar local descriptors into clusters, with each cluster centre acting as a "visual word." This converts variable-length descriptor sets into fixed-length histograms suitable for classification.

Descriptors were collected from up to 2000 training images to ensure diversity while keeping computation manageable. MiniBatch K-Means was used due to its scalability with large descriptor sets.

Parameters:

- $K = 200$
- $\text{batch_size} = 2048$
- $\text{max_iter} = 200$

A vocabulary size of 200 provides a good balance between representational capacity and efficiency. SIFT descriptors, being continuous, generally cluster more coherently than ORB descriptors, so the vocabulary is expected to capture more meaningful structure when built using SIFT.

4.3. Bag-of-Words Histogram Encoding

Each image was encoded into a fixed-length vector by assigning every descriptor to the nearest visual word. This produces a 200-dimensional histogram describing how frequently each visual word appears in the image.

Both hard assignment (single nearest cluster) and soft assignment (top-2 weighted clusters) were evaluated. Soft assignment can mitigate quantisation noise, especially for binary descriptors like ORB, while hard assignment is faster and remains standard in many BoW pipelines.

4.4. Normalisation

Since different images produce different numbers of descriptors, raw histograms are not directly comparable. Therefore, I normalised all BoW histograms using L2 normalisation, scaling each vector to unit length. This prevents images with more detected features from dominating the classifier and leads to more stable SVM optimisation.

4.5. SVM Classification

I performed classification using a linear Support Vector Machine (LinearSVC). Linear SVMs are well suited for high-dimensional histogram-based descriptors such as Bag-of-Words (BoW) features and provide efficient large-scale training.

Hyperparameters tested:

- $C \in \{1.0, 10.0\}$

The parameter C controls the strictness of the margin: lower values enforce a wider, more regularised margin, while higher values allow the classifier to fit the data more tightly by penalising misclassifications more strongly. Testing both settings reveals how sensitive each dataset is to underfitting or overfitting. Performance was evaluated using accuracy, macro precision, macro recall, and macro F1-score, ensuring equal weighting across all classes.

4.6. Hyperparameter Grid Search

I conducted a structured grid to optimise the main components influencing descriptor density, vocabulary behaviour, and classifier regularisation.

Parameters evaluated:

- $\text{step} \in \{4, 8, 12\}$
- $\text{patch_size} \in \{8, 16\}$
- $\text{use_soft} \in \{\text{False}, \text{True}\}$
- $\text{max_dict_images} = 200$
- $C \in \{1.0, 10.0\}$

Step size and patch size determine the granularity of the descriptors and were tuned jointly to account for the large differences in resolution between datasets. I included soft assignment to test whether smoothing improves classification, especially for ORB. The dictionary was built from a fixed set of images for computational efficiency, and varying the SVM's C parameter allowed assessing the appropriate level of regularisation for each dataset. Together, these choices represent the key design trade-offs.

5. CNN Experiments: Overview

This section presents all CNN-based experiments conducted for the project. Each experiment was run independently on CIFAR-100, RGB-D Objects, and iCubWorld, enabling a controlled comparison of how different deep learning strategies behave under varying visual complexity, resolution, and noise. All datasets used a consistent 70%/15%/15% train/validation/test split. The validation set was used exclusively for hyperparameter selection, early stopping, and architecture comparison.

I experimented progressively from simple to highly sophisticated models:

- **A custom CNN (Mini-VGG style) trained from scratch:** a lightweight baseline used to study architectural choices without transfer learning.
- **Systematic fine-tuning of two ImageNet-pretrained backbones (ResNet-18 and MobileNetV2):** incorporating controlled unfreezing strategies, a structured hyperparameter search, and HPC-based parallel experimentation.
- **A novel two-backbone ensemble model:** combining fine-tuned ResNet-18 and MobileNetV2 feature spaces into a unified classifier, inspired by ensemble-transfer learning research.

5.1. Custom CNN (MiniVGG) Architecture and Rationale

For the first deep learning experiment, I built a custom CNN inspired by VGG-style architectures [11], which rely on repeated 3×3 convolutions and strong regularisation. My goal with this model was to establish a fully trainable baseline and assess how architectural depth, optimisation settings, and dropout influence performance.

I resized all images to 128×128 and applied standard normalisation. I deliberately avoided augmentation here, as I wanted to isolate the direct effect of architectural choices without introducing extra variability.

Architecture

The network I designed consists of:

- **Input layer** ($128 \times 128 \times 3$).
- **1–3 convolutional blocks**, each containing:
 - Two 3×3 Conv2D layers (32 filters) with ReLU activation [21].
 - A 2×2 MaxPooling layer to reduce spatial dimensions.
 - A Dropout layer [12] to improve regularisation.

Increasing the number of blocks increases representational capacity, but also raises the risk of overfitting.

Fully connected head:

- Flatten layer
- Dense(128) with ReLU activation
- Dropout
- Dense(num_classes) with Softmax for classification

This architecture mirrors classic early CNN designs while remaining lightweight enough for efficient training from scratch. It allowed me to systematically study how depth and regularisation affect performance before moving on to more sophisticated transfer learning models.

Hyperparameter Search

To understand how different architectural and training choices influence the model, I carried out a structured grid search over the following hyperparameters:

- number of convolutional blocks: [1, 2, 3]
- learning rate: [1×10^{-3} , 3×10^{-4} , 1×10^{-4}]
- batch size: [32, 64]
- dropout rate: [0.0, 0.3, 0.5]
- optimiser: Adam or SGD (momentum = 0.9)

Varying the number of blocks allowed me to test how much depth the model could use before overfitting. The learning rate determines how quickly the weights move during optimisation, so testing several values helped identify a stable range. Batch size influences the smoothness of the gradient updates, and different dropout rates let me observe how much regularisation the model needed. I compared Adam which typically converges faster with SGD, which can sometimes generalise better.

5.2. Fine-Tuning Pretrained Networks (ResNet-18 and MobileNetV2)

To build stronger models, I fine-tuned two ImageNet-pretrained architectures: ResNet-18 [15] and MobileNetV2 [25]. Transfer learning leverages features learned from large datasets, but its effectiveness depends heavily on hyperparameters and how many layers are unfrozen. I was inspired by findings from SpotTune [26] and the fine-tuning optimisation study by Wojciuk et al. [42].

Because fine-tuning combines many interacting hyperparameters, all large-scale searches were executed on the University of Manchester HPC cluster. [41]

Model Structure

Input preprocessing:

- Images resized to 224×224 (ImageNet standard).
- Random horizontal flipping for mild augmentation.

Backbones:

- ResNet-18 with 4 residual stages.
- MobileNetV2 with inverted residual blocks.

The original 1000-class classifier head was replaced with a new dropout layer (0.1 or 0.3) followed by a linear classification layer.

Fine-Tuning Strategies

Two strategies were tested on each model:

- **Random unfreezing (SpotTune-inspired):** [26] randomly unfreezing k blocks per experiment, with different runs unfreezing different combinations.
- **Last- k unfreezing (conventional):** only the last k blocks are unfrozen to adapt deeper, task-specific features.

For ResNet-18, unfreezing all 4 blocks means the entire network is trainable. For MobileNetV2, unfreezing all 4 inverted residual block clusters also corresponds to unfreezing the entire network.

Hyperparameter Search

Following recommendations from Guo et al. [26] and Wojciuk et al. [42], short 2-epoch runs were used to efficiently explore:

- learning rate: [1×10^{-3} , 5×10^{-4} , 1×10^{-4}]
- dropout: [0.1, 0.3]
- number of unfrozen blocks $k \in \{1, 2, 3, 4\}$
- optimiser: Adam, SGD, RMSprop
- weight decay: [0.0, 1×10^{-4}]

Batch size was fixed to 64 for stability.

Full Training with Best Hyperparameters

After selecting optimal hyperparameters, full training was conducted: 10 epochs, early stopping based on validation accuracy, best checkpoint saved for testing.

This produced four fully fine-tuned models: ResNet-18 (random unfreezing), ResNet-18 (last- k unfreezing), MobileNetV2 (random unfreezing), MobileNetV2 (last- k unfreezing).

5.3. Ensemble Model: Combining ResNet-18 and MobileNetV2

After completing all individual fine-tuning experiments, I developed a novel ensemble model that merges the representations learned by ResNet-18 and MobileNetV2. This design is motivated by Korzh et al. [24], who demonstrated that independently fine-tuned CNNs produce complementary features that improve classification when combined.

Ensemble Architecture

Feature extraction: In the ensemble, each backbone processes the input image independently. I apply global average pooling to compress each feature map into a compact descriptor:

- ResNet-18 $\rightarrow$ 512-dimensional feature vector
- MobileNetV2 $\rightarrow$ 1280-dimensional feature vector

Using pooled vectors avoids spatial misalignment issues and ensures both networks contribute equally regardless of their internal architecture.

Feature fusion: I concatenate the two descriptors into a single 1792-dimensional joint representation. Concatenation preserves the full expressive power of each backbone, allowing the classifier to learn how to weight or combine their contributions rather than forcing premature interaction between the feature spaces.

Classifier head: The fusion vector is passed through a lightweight multilayer perceptron:

- Linear($1792 \rightarrow 512$)
- ReLU activation
- Dropout (0.3) for regularisation
- Linear($512 \rightarrow \text{num_classes}$)

I intentionally keep this head small so that the ensemble emphasises backbone features rather than overfitting to the training set through an overly expressive classifier.

Training Modes

Two modes were evaluated:

- **Classifier-only training:** both backbones frozen; only the fusion classifier trains.
- **Last-block fine-tuning:** unfreezing ResNet layer4 and the final inverted residual block of MobileNetV2.

Training Settings

All ensemble configurations were trained for 5 epochs with:

- Adam optimizer
- learning rate 1×10^{-3}
- weight decay 1×10^{-4}
- batch size 64

Ensemble training converges quickly because the backbones are already fine-tuned, a behaviour also reported by Korzh et al. [24].

6. Results: Classical Computer Vision Approach

6.1. CIFAR-100

For CIFAR-100, the best performance was achieved using SIFT with **step = 8, patch = 8, soft assignment enabled, dictionary size $k = 200$ and $C = 1.0$** , reaching **21.9% accuracy** and **17.4% F1**. ORB performed substantially worse, with a maximum accuracy of **9.5%**. These values already indicated that both descriptors struggle on this dataset.

Impact of Dictionary Size (k). Increasing k from 100 to 200 consistently improved results for both descriptors. For SIFT, accuracy increased from roughly 18% to 22%, while ORB rose from 7.3% to 9.5%. This suggests that even small increases in representational capacity help compensate for the weak features present, although the

improvements remain modest, indicating that the dataset structure itself is the main limitation.

Impact of Step Size. The step size had one of the strongest effects. Increasing the step from 8 to 12 reduced accuracy by 20–30% across both descriptors. Because CIFAR-100 images contain extremely limited fine-grained structure, skipping patches removes much of the already scarce discriminative information, confirming that dense sampling is essential for tiny images.

Impact of Patch Size. Increasing patch size from 8 to 16 degraded performance by 10–15%. Larger patches blur object boundaries and combine unrelated textures, which is especially harmful on small images where objects occupy few pixels. This behaviour indicates that both SIFT and ORB require very local spatial windows to extract any meaningful gradients.

Impact of Soft Assignment. Soft assignment consistently improved accuracy by 1–2.5%. For SIFT, soft assignment reduces quantisation noise in the histogram; for ORB, it stabilises binary cluster assignments. Although the gains were small, they were systematic.

Impact of SVM Regularisation (C). Varying C between 1 and 10 had negligible effect (changes $< 0.3\%$). This suggests that the classifier is not the bottleneck: the descriptors themselves fail to form discriminative visual-word clusters.

Interpretation. Overall, CIFAR-100 confirmed that classical descriptors are inherently mismatched to very low-resolution, cluttered images. Even the best hyperparameters cannot compensate for the lack of texture and limited spatial support available to local features.

6.2. iCubWorld

For iCubWorld, the best SIFT configuration ($k = 200$, step=8, patch=8, soft assignment, $C = 10$) achieved **97.7–98.0% accuracy**, while the best ORB configuration achieved approximately **70%**. These results were dramatically better than those on CIFAR-100.

Impact of Dictionary Size. Increasing k from 100 to 200 produced substantial gains for both descriptors, raising SIFT accuracy from ~ 0.90 to ~ 0.97 and ORB from ~ 0.60 to ~ 0.70 . The larger, more detailed images allow descriptors to form more meaningful clusters, making vocabulary granularity much more important than in CIFAR-100.

Impact of Step Size. Step size of 8 consistently outperformed step size of 12 by 2–15%. Even though the images are larger, dense sampling improves contour coverage and reduces the risk of missing important gradients.

Hyperparameters with Minimal Effect. Patch size (8 vs. 16) and SVM parameter C showed minimal influence, indicating that once the images contain sufficiently rich structure, the descriptors themselves provide enough discriminative information.

Interpretation. iCubWorld demonstrated that when the environment resembles a realistic robotic scenario: single objects, controlled viewpoints, clean backgrounds; classical descriptors can be highly effective. SIFT, in particular, achieved performance comparable to small CNN models.

6.3. RGB-D Object Dataset

The RGB-D dataset produced the most surprising outcomes. Several configurations for both descriptors achieved **perfect 1.0 accuracy**. This was the first time in any of my vision experiments (or previous NLP work) where I obtained flawless performance. I initially suspected a bug and rechecked the preprocessing, BoW encoding, SVM training, and data splits. Everything was correct.

Hyperparameter Effects. Unlike the other datasets, hyperparameters had little impact. With $k = 100$ or $k = 200$, step sizes 8 or 12, patch sizes 8 or 16, soft or hard assignment, and both $C = 1$ or $C = 10$, accuracies remained between 0.97 and 1.00. This invariance indicates that the dataset’s structure dominates performance.

Interpretation. Manual inspection showed that many images are nearly identical, captured on a turntable with fixed lighting and clean backgrounds. Under such conditions, SIFT and ORB descriptors are highly repeatable, and the resulting histograms cluster perfectly in feature space, making classification trivial for a linear SVM.

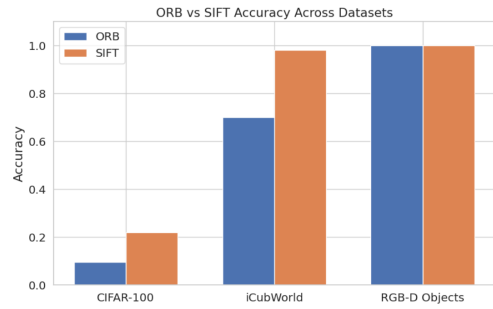


Figure 1. Test accuracies of ORB and SIFT across CIFAR-100, iCubWorld, and RGB-D.

6.4. Cross-Dataset Summary

Across all datasets, a clear trend emerged: as visual structure becomes more controlled and repeatable, classical descriptors transition from *unusable* to *competitive* to *perfect*. This clarified exactly where classical CV remains viable in robotics and where deep learning becomes indispensable.

Table 1 summarises how each hyperparameter influences test accuracy across all datasets, showing that their impact is substantial on CIFAR-100 and iCubWorld, especially for dictionary and step size, while becoming negligible on RGB-D due to the dataset’s controlled visual structure.

Table 1. Hyperparameter Impact on Test Accuracy (Δ in %)

	C-ORB	C-SIFT	I-ORB	I-SIFT	R-ORB	R-SIFT
k size	+1.2	+3.3	+10.2	+5.6	+0.4	0.0
Step size	+1.8	+1.0	+20.5	+1.5	+0.4	0.0
Patch size	0.0	+2.1	0.0	+0.2	0.0	0.0
Soft assign	+1.0	+1.5	+5.1	+0.8	+0.2	+0.1
SVM C	+0.2	+0.2	+0.7	+0.8	0.0	0.0

Future or Alternative Simulations. Several robotics-oriented extensions could further validate these conclusions. (1) Introducing viewpoint, lighting, or background variability into iCubWorld or RGB-D would test how quickly classical descriptors degrade as conditions move away from the “ideal” robot-perception regime. (2) Evaluating larger codebooks (e.g. $k = 400$ or $k = 800$) could show whether SIFT continues to scale in more realistic robotic datasets or whether the performance gains saturate. (3) Adding spatial pyramid matching would preserve coarse object layout, which is often informative when robots observe objects on tables, shelves, or bins. (4) Testing multi-scale SIFT or affine-covariant features would better approximate the viewpoint changes experienced by a moving robot. (5) Finally, evaluating PCA-compressed descriptors could reveal whether reduced-dimensional embeddings allow faster onboard inference without sacrificing accuracy. Together, these simulations would help determine how well classical descriptors generalise to real robotic conditions and whether their observed strengths persist under more realistic operational variability.

7. Results: CNN for Vision

7.1. Results: Custom CNN

Across all three datasets, the CNN experiments revealed clear and highly consistent hyperparameter behaviours. The most influential parameter was the **learning rate**: values of **0.001** and **0.0003** enabled stable convergence for Adam across all datasets, while lower rates (0.0001) caused the model, especially with SGD, to fail to learn entirely, collapsing to near-chance accuracy. This mirrors the expected effect of the learning rate: too small and the network barely updates; too large and optimisation becomes unstable. **Batch size**, which controls how many samples are processed per step, had only a mild effect: both 32 and 64 yielded comparable performance, with slightly smoother learning at 64 on iCub but no systematic advantage on CIFAR-100. **Dropout** behaved as intended: moderate values (0.3) offered mild regularization on CIFAR-100 but provided no benefit in the highly

structured iCub datasets, while high dropout (0.5) occasionally reduced accuracy by suppressing too much feature information. The **choice of optimizer** produced the largest performance gap. **Adam consistently converged**, achieving around **55–65%** on CIFAR-100, **96–98%** on iCub RGB, and **essentially 100% on iCub RGB-D**. In contrast, **SGD was extremely sensitive to hyperparameters**: it achieved reasonable accuracy only when the learning rate was high enough, and otherwise frequently collapsed to **10–20%**, especially on iCub and RGB-D, where small learning rates prevented it from escaping the initial flat region of the loss surface. Importantly, **network depth (1–3 blocks)** had only a minor influence: deeper networks did not outperform shallower ones and sometimes performed worse due to over-regularization or optimization instability. Overall, the hyperparameter search showed that **Adam with a learning rate of 0.001 or 0.0003, batch sizes of 32–64, and dropout ≤ 0.3** provides stable performance across all datasets, while SGD requires careful tuning and remains unreliable for small robotic datasets.

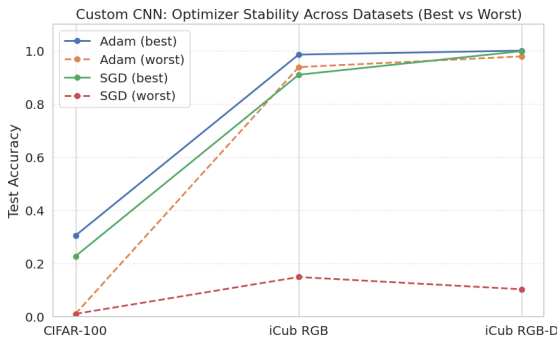


Figure 2. Best and worst CNN test accuracies for Adam and SGD across CIFAR-100, iCub RGB, and iCub RGB-D, illustrating optimizer stability and sensitivity to hyperparameters.

7.2. Results: Fine-Tuned Models (Transfer Learning)

Across all datasets (**iCub**, **RGB-D**, **CIFAR-100**), finetuning pretrained MobileNet and ResNet models produced accuracies ranging from **1.00** down to **0.07**. Such a wide spread highlights how strongly performance depends on learning rate, optimizer choice, regularisation, and the depth of finetuning. Stable settings consistently yielded excellent results, while small deviations often caused catastrophic collapse.

7.2.1. iCub

iCub proved the easiest domain. When hyperparameters were reasonable, both models reached near-perfect accuracy, with **ResNet-Last** the most stable: almost always **0.97–1.00** for $\text{Lr } 10^{-3}$ – 10^{-4} and dropout ≤ 0.3 . Random-block variants of ResNet also achieved **1.00**, but with much higher variance, sometimes collapsing to **0.086**. MobileNet-Last typically achieved **0.85–0.95**, while MobileNet-Random often failed (**0.10–0.20**).

Collapses mainly occurred in two situations: (1) **very small learning rates** (10^{-5}), which prevented meaningful adaptation and left ImageNet features misaligned with iCub; and (2) **randomly unfreezing early/mid blocks**, which disrupted the pretrained hierarchy and destabilised optimisation. Overall, iCub requires only light adaptation, and preserving the pretrained structure: e.g. with Last-Block finetuning was consistently the safest option.

7.2.2. RGB-D Results

The RGB-D dataset produced strong results under stable settings but exhibited sharp collapses under unstable ones. With well-chosen hyperparameters, **ResNet-Last consistently reached 0.97–0.99 accuracy**, while **MobileNet-Last typically achieved 0.90–0.95**. Random-block finetuning showed far greater variance, ranging from **0.20 to 0.98**, depending on which block was reinitialised and the learning rate used.

Learning rate remained the dominant factor. $\text{Lr} = 10^{-3}$ was the most reliable, yielding **0.94–0.99** across both backbones. Reducing it to 5×10^{-4} preserved high accuracy for ResNet (often **0.96–0.98**) but caused occasional drops to **0.85–0.90** in MobileNet. In contrast, 10^{-2} almost always led to divergence, with accuracies falling to **0.10–0.40**. Very small LR (10^{-4} or 10^{-5}) worked with Adam but caused SGD to collapse to **0.07–0.30**.

Regularisation also played a noticeable role. Even mild weight decay (10^{-4}) reduced performance by **1–5 points**, typically lowering ResNet from **0.97** to **0.92** and MobileNet from **0.94** to **0.88**. Similarly, unfreezing more than **2 blocks** degraded accuracy sharply: while **1–2 unfrozen blocks** achieved **0.95–0.99**, unfreezing **3–4 blocks** often led to **0.40–0.85**, especially when combined with higher learning rates.

Architecturally, ResNet proved substantially more stable, rarely falling below **0.95** under reasonable settings, whereas MobileNet varied widely (**0.30–0.96**) due to the sensitivity of depthwise convolutions during finetuning.

Overall, RGB-D rewarded conservative finetuning: moderate learning rates, no weight decay, and shallow unfreezing produced near-perfect results, while aggressive updates caused large performance collapses.

7.2.3. CIFAR-100

CIFAR-100 was the most challenging dataset and the one where finetuning mattered most. The from-scratch CNN reached only **55–65%**, whereas pretrained backbones achieved: MobileNet **65–67%**, ResNet up to **75.2%**, and earlier fixed-block finetuning **80–82%**. This confirms a **+10–15 point** improvement from ImageNet features.

Learning rate dominated behaviour. $\text{Lr} = 10^{-4}$ produced the best results (**70–75%**); $\text{Lr} = 10^{-3}$ was adequate (**60–66%**); $\text{Lr} = 10^{-2}$ became unstable (**0.02–72%**); and $\text{Lr} = 10^{-5}$ led to near-random predictions (**0.5–5%**). Adam handled these regimes most reliably, achieving the best overall result (**75.2%**), while RMSProp was slightly less stable and SGD ranged from **0.4%** to **72.3%** depending on the learning rate.

CIFAR-100 required deeper adaptation: unfreezing **3–4 blocks** gave the strongest results, while only one block often yielded **30–40%**. Dropout had a narrow optimal range (0.1–0.2); at 0.3, especially with SGD, models occasionally collapsed to **2–3%**. Random-Block training matched Last-Block only in its best cases but was far less stable, again showing that preserving hierarchical structure is crucial when data are limited and resolution is low.

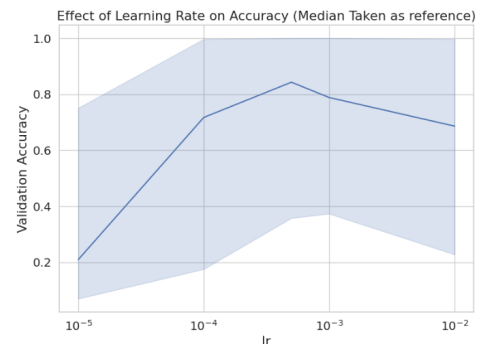


Figure 3. Median validation accuracy across all datasets and models for each hyperparameter value, illustrating global stability trends and collapse regimes in the full finetuning search space.

Hyperparameter	Best Acc.	Worst Acc.	Range
Learning Rate	0.7478	0.0103	0.7375
Optimizer	0.7478 (Adam)	0.0008 (SGD)	0.7470
Dropout	0.7478 (0.3)	0.0022 (0.1)	0.7456
Weight Decay	0.7478 (0.0001)	0.0069 (0.0001)	0.7409
Blocks Unfrozen (k)	0.7478 (4 blocks)	0.0103 (1 block)	0.7375

Table 2. Best and worst CIFAR-100 ResNet Last-Block finetuning accuracies grouped by hyperparameter family. Values are computed from all runs in the provided dataset.

7.3. Ensemble Results Across the Three Datasets

Because the individual fine-tuned models were already highly optimised, the ensemble had very limited room to improve performance.

iCub and RGB-D. Both datasets reached **1.00 accuracy and macro F1** with single models, and the ensemble matched but could not exceed this ceiling. The models were already saturated, leaving no opportunity for ensembling to reveal additional complementary information.

CIFAR-100. CIFAR-100 is more challenging, so the ensemble produced a small improvement: from **75.2%** (best single model) to **76.1%**.

This modest **+0.9 pp** gain is expected because the fine-tuned models were still strong and highly correlated, limiting ensemble diversity. Overall, the ensembles confirm that finetuning already produced models near their maximum achievable performance. Only CIFAR-100, where accuracy was not saturated, showed a slight benefit, while the simpler RGB-D and iCub datasets left no space for improvement.

7.4. Future Directions and Alternative Simulations

Several additional simulations could further strengthen the interpretation of the CNN results. Testing **transformer-based models (e.g., ViT, DeiT)** would reveal whether architectures with global attention exhibit the same collapse-prone behaviour seen in CNN finetuning. Incorporating **larger-capacity backbones** such as EfficientNet or ConvNeXt could show whether scaling improves stability or simply increases sensitivity to learning rate. Introducing **true architectural diversity** would also make ensembles more informative by reducing prediction correlation. Exploring **progressive or discriminative layer-unfreezing schedules** could test whether smoother adaptation prevents the abrupt degradation observed with random or deep unfreezing. Finally, evaluating finetuning under **low-data or few-shot regimes** would mirror realistic robotic constraints and clarify how brittle each hyperparameter becomes when data is scarce.

8. Comparison of Deep Learning and Classical Computer Vision Approaches

The two recognition pipelines differ not only in performance but in the **nature of the visual information** they are able to exploit. Classical descriptors depend on the presence of **stable, high-frequency gradients** at specific image locations, whereas CNNs build **multi-scale learned representations** that integrate context, suppress noise, and reshape features through training. The results across the three datasets reflect these underlying mechanisms clearly.

On **CIFAR-100**, classical descriptors performed poorly because the images provide almost none of the structural cues they rely on: textures collapse at 32×32 resolution, edges blur into single pixels, and unstable keypoints make the BoW histograms dominated by quantisation noise. No hyperparameter could overcome this, because the issue is fundamentally that the **signal classical methods require does not exist**. Deep models succeeded because CNN filters aggregate information over increasingly large receptive fields, allowing them to infer **coarse shapes, colour distributions, and category-level structure** even when no reliable local gradients are present.

In **iCubWorld**, the relationship shifts. The robot-centric acquisition process produces images with clear contours, large objects, and limited background clutter. Here SIFT obtains thousands of repeatable gradient patches per image, and these patches correlate strongly with class identity. Classical descriptors succeed not because they are inherently powerful, but because **the environment aligns with their assumptions**: objects are centred, illumination is stable, and viewpoint variation is modest. Deep learning still offers higher stability and typically higher peak performance, but its architectural advantages matter less when the input already contains clean visual structure.

The **RGB-D dataset** strengthens this observation. The turntable acquisition produces images that are nearly identical across frames; keypoints reappear in the same locations with the same orientations, and BoW histograms form perfectly separable clusters. In this regime, the learning problem becomes almost trivial: the dataset provides enough redundancy that **both classical descriptors and CNNs achieve near-perfect accuracy** for essentially the same reason.

Across all datasets, the central distinction is that **deep learning reshapes the feature space to create discriminative structure**, while **classical methods rely entirely on structure already present in the raw data**. When that structure is weak or noisy (CIFAR-100), deep models dominate. When it is strong, repeatable, and tightly controlled (iCubWorld, RGB-D), classical descriptors remain viable and sometimes nearly optimal. For robotic perception, this implies that classical pipelines can be highly effective under controlled sensing conditions, whereas deep learning is indispensable whenever the environment becomes cluttered, unpredictable, or characterised by low-detail or weak visual structure.

References

- [1] L. Weiss, A. Sanderson, and C. Neuman, "Dynamic sensor-based control of robots with visual feedback", *IEEE Journal on Robotics and Automation*, vol. 3, no. 5, pp. 404–417, 2003.
- [2] N. Dalal and B. Triggs, "Histograms of oriented gradients for human detection", in *2005 IEEE computer society conference on computer vision and pattern recognition (CVPR'05)*, IEEE, vol. 1, 2005, pp. 886–893.
- [3] H. Bay, T. Tuytelaars, and L. Van Gool, "Surf: Speeded up robust features", in *European conference on computer vision*, Springer, 2006, pp. 404–417.
- [4] J. Yang, Y.-G. Jiang, A. G. Hauptmann, and C.-W. Ngo, "Evaluating bag-of-visual-words representations in scene classification", in *Proceedings of the international workshop on Workshop on multimedia information retrieval*, 2007, pp. 197–206.
- [5] D. Kragic, M. Vincze, et al., "Vision for robotics", *Foundations and Trends® in Robotics*, vol. 1, no. 1, pp. 1–78, 2009.
- [6] A. Krizhevsky, G. Hinton, et al., "Learning multiple layers of features from tiny images", 2009.
- [7] K. Lai, L. Bo, X. Ren, and D. Fox, "A large-scale hierarchical multi-view rgb-d object dataset", in *2011 IEEE international conference on robotics and automation*, IEEE, 2011, pp. 1817–1824.
- [8] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks", *Advances in neural information processing systems*, vol. 25, 2012.
- [9] T. Lindeberg, "Scale invariant feature transform", 2012.
- [10] S. Fanelli et al., "Icub world: Friendly robots help building good vision data-sets", in *Proceedings of the IEEE conference on computer vision and pattern recognition workshops*, 2013, pp. 700–705.
- [11] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition", *arXiv preprint arXiv:1409.1556*, 2014.
- [12] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting", *The journal of machine learning research*, vol. 15, no. 1, pp. 1929–1958, 2014.
- [13] I. Lenz, H. Lee, and A. Saxena, "Deep learning for detecting robotic grasps", *The International Journal of Robotics Research*, vol. 34, no. 4-5, pp. 705–724, 2015.
- [14] J. Dai, Y. Li, K. He, and J. Sun, "R-fcn: Object detection via region-based fully convolutional networks", *Advances in neural information processing systems*, vol. 29, 2016.
- [15] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition", in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.
- [16] F. N. Iandola, S. Han, M. W. Moskewicz, K. Ashraf, W. J. Dally, and K. Keutzer, "Squeezenet: Alexnet-level accuracy with 50x fewer parameters and < 0.5 mb model size", *arXiv preprint arXiv:1602.07360*, 2016.
- [17] T. B. Sheridan, "Human-robot interaction: Status and challenges", *Human factors*, vol. 58, no. 4, pp. 525–532, 2016.
- [18] K. He, G. Gkioxari, P. Dollár, and R. Girshick, "Mask r-cnn", in *Proceedings of the IEEE international conference on computer vision*, 2017, pp. 2961–2969.
- [19] R. Mur-Artal and J. D. Tardós, "Visual-inertial monocular slam with map reuse", *IEEE Robotics and Automation Letters*, vol. 2, no. 2, pp. 796–803, 2017.
- [20] C. R. Qi, H. Su, K. Mo, and L. J. Guibas, "Pointnet: Deep learning on point sets for 3d classification and segmentation", in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2017, pp. 652–660.
- [21] A. F. Agarap, "Deep learning using rectified linear units (relu)", *arXiv preprint arXiv:1803.08375*, 2018.
- [22] R.-J. Halme, M. Lanz, J. Kämäräinen, R. Pieters, J. Latokartano, and A. Hietanen, "Review of vision-based safety systems for human-robot collaboration", *Procedia Cirp*, vol. 72, pp. 111–116, 2018.
- [23] D. Kalashnikov et al., "Scalable deep reinforcement learning for vision-based robotic manipulation", in *Conference on robot learning*, PMLR, 2018, pp. 651–673.
- [24] O. Korzh, M. Joaristi, and E. Serra, "Convolutional neural network ensemble fine-tuning for extended transfer learning", in *International Conference on Big Data*, Springer, 2018, pp. 110–123.
- [25] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "Mobilenetv2: Inverted residuals and linear bottlenecks", in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2018, pp. 4510–4520.
- [26] Y. Guo, H. Shi, A. Kumar, K. Grauman, T. Rosing, and R. Feris, "Spottune: Transfer learning through adaptive fine-tuning", in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2019, pp. 4805–4814.
- [27] N. O'Mahony et al., "Deep learning vs. traditional computer vision", in *Science and information conference*, Springer, 2019, pp. 128–144.
- [28] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, "End-to-end object detection with transformers", in *European conference on computer vision*, Springer, 2020, pp. 213–229.
- [29] A. Dosovitskiy, "An image is worth 16x16 words: Transformers for image recognition at scale", *arXiv preprint arXiv:2010.11929*, 2020.
- [30] A. I. Károlyi, P. Galambos, J. Kutí, and I. J. Rudas, "Deep learning in robotics: Survey on model structures and training strategies", *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, vol. 51, no. 1, pp. 266–279, 2020.
- [31] X. Zhu, W. Su, L. Lu, B. Li, X. Wang, and J. Dai, "Deformable detr: Deformable transformers for end-to-end object detection", *arXiv preprint arXiv:2010.04159*, 2020.
- [32] L. Alzubaidi et al., "Review of deep learning: Concepts, cnn architectures, challenges, applications, future directions", *Journal of big Data*, vol. 8, no. 1, p. 53, 2021.
- [33] A. Radford et al., "Learning transferable visual models from natural language supervision", in *International conference on machine learning*, PMLR, 2021, pp. 8748–8763.
- [34] Z. Teed and J. Deng, "Droid-slam: Deep visual slam for monocular, stereo, and rgb-d cameras", *Advances in neural information processing systems*, vol. 34, pp. 16 558–16 569, 2021.
- [35] B. Ali, R. Sadekov, and V. Tsodokova, "A review of navigation algorithms for unmanned aerial vehicles based on computer vision systems", *Gyroscopy and Navigation*, vol. 13, no. 4, pp. 241–252, 2022.
- [36] K. Han et al., "A survey on vision transformer", *IEEE transactions on pattern analysis and machine intelligence*, vol. 45, no. 1, pp. 87–110, 2022.
- [37] A. Buhmann and C. Fieseler, "Deep learning meets deep democracy: Deliberative governance and responsible innovation in artificial intelligence", *Business Ethics Quarterly*, vol. 33, no. 1, pp. 146–179, 2023.
- [38] D. Driess et al., "Palm-e: An embodied multimodal language model", 2023.
- [39] M. N. Favorskaya, "Deep learning for visual slam: The state-of-the-art and future trends", *Electronics*, vol. 12, no. 9, p. 2006, 2023.
- [40] A. Y. Wang, K. Kay, T. Naselaris, M. J. Tarr, and L. Wehbe, "Better models of human high-level visual cortex emerge from natural language supervision with a large and diverse dataset", *Nature Machine Intelligence*, vol. 5, no. 12, pp. 1415–1426, 2023.
- [41] Research IT, University of Manchester. "The computational shared facility (csf)". Accessed: 2025-02-07. [Online]. Available: <https://research-it.manchester.ac.uk/services/the-computational-shared-facility-csf/>.
- [42] M. Wojciuk, Z. Swiderska-Chadaj, K. Siwek, and A. Gertych, "Improving classification accuracy of fine-tuned cnn models: Impact of hyperparameter optimization", *Heliyon*, vol. 10, no. 5, 2024.

A. Classical Computer Vision Approach

```
1 import torchvision
2 from torchvision import transforms
3 import numpy as np
4 from sklearn.model_selection import train_test_split
5
6 IMG_SIZE = 128
```

```

7
8 transform = transforms.Compose([
9     transforms.Resize((IMG_SIZE, IMG_SIZE)),
10    transforms.ToTensor(),      # gives tensor [0,1]
11 ])
12
13 cifar_train = torchvision.datasets.CIFAR100(
14     root="/content/data", train=True, download=True,
15     ↪ transform=transform
16 )
17 cifar_test = torchvision.datasets.CIFAR100(
18     root="/content/data", train=False, download=True,
19     ↪ transform=transform
20 )
21 data = np.concatenate([
22     np.stack([np.array((img * 255).byte().permute(1,2,0))
23     ↪ for img,_ in cifar_train]),
24     np.stack([np.array((img * 255).byte().permute(1,2,0))
25     ↪ for img,_ in cifar_test])
26 ], axis=0)
27
28 labels = np.concatenate([
29     np.array([lab for _,lab in cifar_train]),
30     np.array([lab for _,lab in cifar_test])
31 ])
32
33 print("Total images:", len(data))
34 print("Num classes:", len(set(labels)))
35
36 # =====
37 # STRATIFIED TRAIN / VAL / TEST SPLIT
38 # =====
39 X_train, X_temp, y_train, y_temp = train_test_split(
40     data, labels, test_size=0.30, stratify=labels,
41     ↪ random_state=42
42 )
43
44 X_val, X_test, y_val, y_test = train_test_split(
45     X_temp, y_temp, test_size=0.50, stratify=y_temp,
46     ↪ random_state=42
47 )
48
49 print("Train:", X_train.shape)
50 print("Val: ", X_val.shape)
51 print("Test: ", X_test.shape)
52 # 2. DENSE SIFT
53 # =====
54
55 def extract_dense_sift(img: np.ndarray, step: int = 4,
56     ↪ patch_size: int = 16):
57     gray = cv2.cvtColor(img, cv2.COLOR_RGB2GRAY)
58     sift = cv2.SIFT_create()
59     h, w = gray.shape
60
61     keypoints = [
62         cv2.KeyPoint(float(x), float(y), patch_size)
63         for y in range(0, h, step)
64         for x in range(0, w, step)
65     ]
66
67     if not keypoints:
68         return None
69
70     _, desc = sift.compute(gray, keypoints)
71     return desc
72
73 # =====
74 # 3. DICTIONARY BUILDING
75 # =====
76
77 def build_dictionary(
78     X,
79     k=200,
80     batch_size=2048,
81     max_iter=200,
82     step=4,
83     patch_size=16,
84     max_images=1000,
85     random_state=42
86 ):
87     rng = np.random.RandomState(random_state)
88
89     n_total = len(X)
90     n_use = min(max_images, n_total)
91
92     indices = rng.choice(n_total, size=n_use, replace=
93     ↪ False)
94
95     sift_desc = []
96
97     print(f"[Dictionary] Extracting SIFT from {n_use}
98     ↪ training images...")
99     for idx in tqdm(indices):
100         img = X[idx]
101         desc = extract_dense_sift(img, step=step,
102         ↪ patch_size=patch_size)
103         if desc is not None:
104             sift_desc.append(desc)
105
106     if len(sift_desc) == 0:
107         raise ValueError("No SIFT descriptors extracted!")
108     ↪ )
109
110     sift_desc = np.vstack(sift_desc)
111
112     kmeans = MiniBatchKMeans(
113         n_clusters=k,
114         batch_size=batch_size,
115         max_iter=max_iter,
116         random_state=random_state
117     ).fit(sift_desc)
118
119     return kmeans
120
121 # =====
122 # 4. SOFT ASSIGNMENT
123 # =====
124
125 def soft_assign(desc, centers, top_k=2):
126     dists = np.linalg.norm(desc[:, None] - centers[None,
127     ↪ :], axis=2)
128     indices = np.argsort(dists, axis=1)[: , :top_k]
129     weights = 1 / (dists[np.arange(len(desc))[:, None],
130     ↪ indices] + 1e-6)
131     weights = weights / weights.sum(axis=1, keepdims=True)
132     ↪ )
133     return indices, weights
134
135 # =====
136 # 5. BoW ENCODING
137 # =====
138
139 def encode_bow(
140     X,
141     kmeans,
142     k=None,
143     step=4,
144     patch_size=16,
145     use_soft=False,
146     soft_top_k=2
147 ):
148     vocab = kmeans.cluster_centers_
149     if k is None:
150         k = vocab.shape[0]
151
152     H = []
153
154     print("[BoW] Encoding histograms for", len(X), "
155     ↪ images...")
156     for img in tqdm(X):
157         desc = extract_dense_sift(img, step=step,
158         ↪ patch_size=patch_size)
159
160         if desc is None:
161             H.append(np.zeros(k, dtype=np.float32))
162             continue
163
164         if use_soft:
165             idx, w = soft_assign(desc, vocab, top_k=
166             ↪ soft_top_k)
167             hist = np.zeros(k, dtype=np.float32)
168             for i in range(len(idx)):
169                 for j in range(idx.shape[1]):
170                     hist[idx[i, j]] += w[i, j]
171
172         else:
173             words = kmeans.predict(desc)
174             hist, _ = np.histogram(words, bins=np.arange
175             ↪ (k+1))

```



```

159         hist = hist.astype(np.float32)
160
161         H.append(hist)
162
163         return np.vstack(H)
164
165     # =====
166     # 6. NORMALIZATION + TF-IDF
167     # =====
168     def normalize_bow_train_val_test(
169         H_train,
170         H_val,
171         H_test,
172         norm_type="l2",
173         use_tfidf=False
174     ):
175         transformer = None
176
177         if use_tfidf:
178             transformer = TfidfTransformer(norm=None)
179             H_train = transformer.fit_transform(H_train).
180             ↪ toarray()
181             H_val = transformer.transform(H_val).toarray()
182             H_test = transformer.transform(H_test).toarray()
183             ↪
184
185         H_train = normalize(H_train, norm=norm_type)
186         H_val = normalize(H_val, norm=norm_type)
187         H_test = normalize(H_test, norm=norm_type)
188
189         return H_train, H_val, H_test, transformer
190
191     # =====
192     # 7. SVM TRAINING + EVALUATION
193     # =====
194     def train_svm(H_train, y_train, C=1.0, kernel="linear"):
195         if kernel == "linear":
196             clf = LinearSVC(C=C, max_iter=10000)
197         else:
198             clf = SVC(C=C, kernel=kernel)
199         clf.fit(H_train, y_train)
200         return clf
201
202     def evaluate_svm(clf, H, y_true):
203         y_pred = clf.predict(H)
204         return {
205             "accuracy": accuracy_score(y_true, y_pred),
206             "precision": precision_score(y_true, y_pred,
207             ↪ average="macro", zero_division=0),
208             "recall": recall_score(y_true, y_pred,
209             ↪ average="macro", zero_division=0),
210             "f1": f1_score(y_true, y_pred, average="
211             ↪ macro", zero_division=0),
212         }
213
214     # =====
215     # 8. FULL PIPELINE
216     # =====
217     def run_pipeline(
218         X_train,
219         X_val,
220         X_test,
221         y_train,
222         y_val,
223         y_test,
224         k=200,
225         step=4,
226         patch_size=16,
227         batch_size=2048,
228         max_iter=200,
229         use_soft=False,
230         use_tfidf=False,
231         norm_type="l2",
232         C=1.0,
233         kernel="linear",
234         max_dict_images=2000,
235         soft_top_k=2,
236         random_state=42
237     ):
238         print("\n=== PIPELINE START ===")
239
240         kmeans = build_dictionary(
241             X_train,

```

```

239         k=k,
240         batch_size=batch_size,
241         max_iter=max_iter,
242         step=step,
243         patch_size=patch_size,
244         max_images=max_dict_images,
245         random_state=random_state
246     )
247
248     H_train = encode_bow(X_train, kmeans, k=k, step=step,
249     ↪ patch_size=patch_size,
250     ↪ use_soft=use_soft, soft_top_k=
251     ↪ soft_top_k)
252     H_val = encode_bow(X_val, kmeans, k=k, step=step,
253     ↪ patch_size=patch_size,
254     ↪ use_soft=use_soft, soft_top_k=
255     ↪ soft_top_k)
256     H_test = encode_bow(X_test, kmeans, k=k, step=step,
257     ↪ patch_size=patch_size,
258     ↪ use_soft=use_soft, soft_top_k=
259     ↪ soft_top_k)
260
261     H_train, H_val, H_test, tfidf =
262     ↪ normalize_bow_train_val_test(
263     ↪ H_train, H_val, H_test, norm_type=norm_type,
264     ↪ use_tfidf=use_tfidf
265     )
266
267     clf = train_svm(H_train, y_train, C=C, kernel=kernel)
268     ↪
269
270     val_metrics = evaluate_svm(clf, H_val, y_val)
271     test_metrics = evaluate_svm(clf, H_test, y_test)
272
273     print("Val:", val_metrics)
274     print("Test:", test_metrics)
275     print("=== PIPELINE END ===")
276
277     return val_metrics, test_metrics, clf, kmeans, tfidf
278
279     # =====
280     # 9. GRID SEARCH
281     # =====
282     def grid_search_bow_sift_svm(
283         X_train,
284         X_val,
285         X_test,
286         y_train,
287         y_val,
288         y_test,
289         param_grid,
290         base_params=None
291     ):
292         if base_params is None:
293             base_params = {}
294
295         keys = list(param_grid.keys())
296         values = list(param_grid.values())
297         results = []
298
299         for combo in product(*values):
300             params = dict(zip(keys, combo))
301             merged = {**base_params, **params}
302
303             print("\n=====")
304             print("Testing params:", merged)
305
306             val_m, _, clf, kmeans, tfidf = run_pipeline(
307                 X_train, X_val, X_test,
308                 y_train, y_val, y_test,
309                 k=merged.get("k", 200),
310                 step=merged.get("step", 4),
311                 patch_size=merged.get("patch_size", 16),
312                 batch_size=merged.get("batch_size", 2048),
313                 max_iter=merged.get("max_iter", 200),
314                 use_soft=merged.get("use_soft", False),
315                 use_tfidf=merged.get("use_tfidf", False),
316                 norm_type=merged.get("norm_type", "l2"),
317                 C=merged.get("C", 1.0),
318                 kernel=merged.get("kernel", "linear"),
319                 max_dict_images=merged.get("max_dict_images",
320                 ↪ 2000),
321                 soft_top_k=merged.get("soft_top_k", 2),
322                 random_state=merged.get("random_state", 42)
323             )

```

```

314         row = {
315             **merged,
316             "val_accuracy": val_m["accuracy"]
317             # "test_accuracy": test_m["accuracy"]
318         }
319         results.append(row)
320
321     df_results = pd.DataFrame(results)
322     best_idx = df_results["val_accuracy"].idxmax()
323     best_row = df_results.iloc[best_idx]
324
325     return df_results, best_row
326
327 param_grid = {
328     "k": [200],
329     "step": [8, 12],
330     "patch_size": [8, 16],
331     "use_soft": [False, True],
332     "use_tfidf": [False],
333     "C": [1.0, 10.0],
334     "max_dict_images": [200]
335 }
336
337 base_params = {
338     "batch_size": 2048,
339     "max_iter": 200,
340     "norm_type": "l2",
341     "kernel": "linear",
342     "soft_top_k": 2,
343     "random_state": 42,
344 }
345
346 df_results, best_row = grid_search_bow_sift_svm(
347     X_train,
348     X_val,
349     X_test,
350     y_train,
351     y_val,
352     y_test,
353     param_grid=param_grid,
354     base_params=base_params
355 )
356
357 print(df_results)
358 print("\nBest row:\n", best_row)
359
360 df_results.to_csv("bow_sift_grid_results.csv", index=
361     ↪ False)
362
363 # After grid search:
364 best = best_row.to_dict()
365
366 # Retrain dictionary + model on X_train using best
367     ↪ params
368 _, test_metrics, _, _, _ = run_pipeline(
369     X_train, X_val, X_test,
370     y_train, y_val, y_test,
371     k=best["k"],
372     step=best["step"],
373     patch_size=best["patch_size"],
374     use_soft=best["use_soft"],
375     use_tfidf=best["use_tfidf"],
376     C=best["C"],
377     max_dict_images=best["max_dict_images"]
378 )
379
380 print("Final TEST accuracy:", test_metrics["accuracy"])

```

B. Custom CNN

```

1 # =====
2 # 0. IMPORTS
3 # =====
4 import os
5 import numpy as np
6 import pandas as pd
7 from tqdm import tqdm
8
9 from sklearn.model_selection import train_test_split
10 from sklearn.metrics import accuracy_score,
    ↪ precision_score, recall_score, f1_score

```

```

11 import torch
12 import torch.nn as nn
13 import torch.optim as optim
14 from torch.utils.data import DataLoader, Subset
15 from torchvision.datasets import ImageFolder
16 from torchvision.transforms import T
17
18
19 from google.colab import drive
20 drive.mount('/content/drive')
21
22
23 # =====
24 # 1. LOAD DATASET FROM GOOGLE DRIVE
25 # =====
26
27 DATASET_PATH = "/content/drive/MyDrive/rgbd_rgb"
28 IMG_SIZE = 128
29
30 transform = T.Compose([
31     T.Resize((IMG_SIZE, IMG_SIZE)),
32     T.ToTensor()
33 ])
34
35 dataset = ImageFolder(DATASET_PATH, transform=transform)
36 num_classes = len(dataset.classes)
37
38 print("Total images:", len(dataset))
39 print("Classes:", dataset.classes)
40
41
42 # =====
43 # 2. STRATIFIED TRAIN/VAL/TEST SPLIT
44 # =====
45 labels = [label for _, label in dataset.samples]
46
47 train_idx, temp_idx, y_train, y_temp = train_test_split(
48     np.arange(len(dataset)),
49     labels,
50     test_size=0.30,
51     stratify=labels,
52     random_state=42
53 )
54
55 val_idx, test_idx, y_val, y_test = train_test_split(
56     temp_idx,
57     y_temp,
58     test_size=0.50,
59     stratify=y_temp,
60     random_state=42
61 )
62
63 train_set = Subset(dataset, train_idx)
64 val_set = Subset(dataset, val_idx)
65 test_set = Subset(dataset, test_idx)
66
67 print("Train:", len(train_set), " Val:", len(val_set), "
    ↪ Test:", len(test_set))
68
69
70 # =====
71 # 3. MINI-VGG CNN FROM SCRATCH
72 # =====
73
74 class MiniVGG(nn.Module):
75     def __init__(self, num_classes=10, blocks=2, dropout=
    ↪ 0.3):
76         super().__init__()
77
78         layers = []
79         in_channels = 3
80
81         # Create multiple conv blocks
82         for _ in range(blocks):
83             layers += [
84                 nn.Conv2d(in_channels, 32, kernel_size=3,
    ↪ padding=1),
85                 nn.ReLU(),
86                 nn.Conv2d(32, 32, kernel_size=3, padding
    ↪ =1),
87                 nn.ReLU(),
88                 nn.MaxPool2d(2),
89                 nn.Dropout(dropout)
90             ]
91             in_channels = 32

```

```

92         self.features = nn.Sequential(*layers)
93         self.classifier = nn.Sequential(
94             nn.Flatten(),
95             nn.Linear((IMG_SIZE // (2**blocks))*2 * 32,
96                 ↪ 128),
97             nn.ReLU(),
98             nn.Dropout(dropout),
99             nn.Linear(128, num_classes)
100         )
101
102     def forward(self, x):
103         x = self.features(x)
104         return self.classifier(x)
105
106
107 # =====
108 # 4. TRAINING LOOP
109 # =====
110
111 def train_one_epoch(model, loader, criterion, optimizer,
112     ↪ device):
113     model.train()
114     total_loss = 0
115     for X, y in loader:
116         X, y = X.to(device), y.to(device)
117         optimizer.zero_grad()
118         pred = model(X)
119         loss = criterion(pred, y)
120         loss.backward()
121         optimizer.step()
122         total_loss += loss.item()
123     return total_loss / len(loader)
124
125 def evaluate(model, loader, device):
126     model.eval()
127     preds, labels = [], []
128     with torch.no_grad():
129         for X, y in loader:
130             X, y = X.to(device), y.to(device)
131             out = model(X)
132             pred = out.argmax(dim=1)
133             preds.extend(pred.cpu().numpy())
134             labels.extend(y.cpu().numpy())
135
136     return {
137         "accuracy": accuracy_score(labels, preds),
138         "precision": precision_score(labels, preds,
139             ↪ average="macro", zero_division=0),
140         "recall": recall_score(labels, preds, average="
141             ↪ macro", zero_division=0),
142         "f1": f1_score(labels, preds, average="macro",
143             ↪ zero_division=0)
144     }
145
146 # =====
147 # 5. HYPERPARAMETER GRID SEARCH
148 # =====
149
150 param_grid = {
151     "blocks": [1, 2, 3],
152     "batch_size": [32, 64],
153     "lr": [1e-3, 3e-4, 1e-4],
154     "dropout": [0.0, 0.3, 0.5],
155     "optimizer": ["adam", "sgd"]
156 }
157
158 results = []
159 device = torch.device("cuda" if torch.cuda.is_available()
160     ↪ else "cpu")
161 criterion = nn.CrossEntropyLoss()
162
163 from itertools import product
164
165 for blocks, batch_size, lr, dropout, opt_name in product(
166     param_grid["blocks"],
167     param_grid["batch_size"],
168     param_grid["lr"],
169     param_grid["dropout"],
170     param_grid["optimizer"]
171 ):
172
173     print(f"\nTesting: blocks={blocks}, batch={batch_size}
174         ↪ }, lr={lr}, dropout={dropout}, opt={opt_name}")
175
176     # Dataloaders
177     train_loader = DataLoader(train_set, batch_size=
178         ↪ batch_size, shuffle=True)
179     val_loader = DataLoader(val_set, batch_size=
180         ↪ batch_size)
181     test_loader = DataLoader(test_set, batch_size=
182         ↪ batch_size)
183
184     # Model
185     model = MiniVGG(num_classes, blocks=blocks, dropout=
186         ↪ dropout).to(device)
187
188     if opt_name == "adam":
189         optimizer = optim.Adam(model.parameters(), lr=lr)
190     ↪
191     else:
192         optimizer = optim.SGD(model.parameters(), lr=lr,
193             ↪ momentum=0.9)
194
195     for epoch in range(5):
196         loss = train_one_epoch(model, train_loader,
197             ↪ criterion, optimizer, device)
198
199     # Evaluate
200     val_metrics = evaluate(model, val_loader, device)
201     test_metrics = evaluate(model, test_loader, device)
202
203     row = {
204         "blocks": blocks,
205         "batch_size": batch_size,
206         "lr": lr,
207         "dropout": dropout,
208         "optimizer": opt_name,
209         "val_acc": val_metrics["accuracy"],
210         "val_prec": val_metrics["precision"],
211         "val_rec": val_metrics["recall"],
212         "val_f1": val_metrics["f1"],
213         "test_acc": test_metrics["accuracy"],
214         "test_prec": test_metrics["precision"],
215         "test_rec": test_metrics["recall"],
216         "test_f1": test_metrics["f1"]
217     }
218
219     results.append(row)
220
221     print(
222         f"RESULT | blocks={blocks}, batch={batch_size}, lr={
223             ↪ lr}, dropout={dropout}, opt={opt_name} "
224         f"| val_acc={val_metrics['accuracy']:.4f}, val_f1={
225             ↪ val_metrics['f1']:.4f}, val_prec = {val_metrics
226             ↪ ['precision']}, val_rec = {val_metrics['recall']}
227         ↪ "
228         f"| test_acc={test_metrics['accuracy']:.4f}, test_f1
229             ↪ = {test_metrics['f1']:.4f}, test_prec = {
230             ↪ test_metrics['precision']}, test_rec = {
231             ↪ test_metrics['recall']}"
232     )
233
234 # =====
235 # 6. SAVE RESULTS TO CSV
236 # =====
237 df = pd.DataFrame(results)
238 df.to_csv("cnn_grid_search_results.csv", index=False)
239 df

```

Code 1. Custom CNN Code

C. Finetuning Models

```

1 import os
2 import json
3 import time
4 from pathlib import Path
5 import random

```



```

6 import itertools
7
8 import numpy as np
9 from sklearn.model_selection import
    ↳ StratifiedShuffleSplit
10 from sklearn.metrics import f1_score
11 import pandas as pd
12
13 import torch
14 import torch.nn as nn
15 from torch.utils.data import Dataset, DataLoader
16 from torchvision import transforms, models
17 from PIL import Image
18
19 # ===== PATHS =====
20 PROJECT_ROOT = Path("/scratch/c00714ij/project")
21 DATASET_ROOT = PROJECT_ROOT / "datasets" / "MyDataset"
22
23 RESULTS_ROOT = PROJECT_ROOT / "results"
24 SPLIT_DIR = PROJECT_ROOT / "splits"
25 MODELS_DIR = PROJECT_ROOT / "models"
26
27 for d in [RESULTS_ROOT, SPLIT_DIR, MODELS_DIR]:
28     d.mkdir(parents=True, exist_ok=True)
29 # =====
30
31 DEVICE = torch.device("cuda" if torch.cuda.is_available()
    ↳ else "cpu")
32 print("Using device:", DEVICE)
33
34 # ----- Dataset scanning & splits
    ↳ -----
35
36 def build_paths_and_labels():
37     classes = sorted([d.name for d in DATASET_ROOT.
    ↳ iterdir() if d.is_dir()])
38     class_to_idx = {cls: i for i, cls in enumerate(
    ↳ classes)}
39
40     paths = []
41     labels = []
42
43     for cls in classes:
44         class_dir = DATASET_ROOT / cls
45         for img_file in class_dir.iterdir():
46             if img_file.suffix.lower() in [".png", ".jpg"
    ↳ , ".jpeg"]:
47                 paths.append(str(img_file))
48                 labels.append(class_to_idx[cls])
49
50     paths = np.array(paths, dtype=str)
51     labels = np.array(labels, dtype=int)
52
53     print("Classes:", classes)
54     print("Total images:", len(paths))
55     return paths, labels, classes, class_to_idx
56
57
58 def make_subset(paths, labels, fraction=0.5, seed=42):
59     np.random.seed(seed)
60     unique_classes = np.unique(labels)
61     selected = []
62
63     for c in unique_classes:
64         idx = np.where(labels == c)[0]
65         n = max(1, int(len(idx) * fraction))
66         chosen = np.random.choice(idx, n, replace=False)
67         selected.extend(chosen)
68
69     selected = np.array(selected, dtype=int)
70     return paths[selected], labels[selected]
71
72
73 def save_split(name, file_paths, file_labels):
74     np.savetxt(SPLIT_DIR / f"{name}_paths.txt",
    ↳ file_paths, fmt="%s")
75     np.savetxt(SPLIT_DIR / f"{name}_labels.txt",
    ↳ file_labels, fmt="%d")
76
77
78 def load_split(name):
79     paths = np.loadtxt(SPLIT_DIR / f"{name}_paths.txt",
    ↳ dtype=str)
80     labels = np.loadtxt(SPLIT_DIR / f"{name}_labels.txt",
    ↳ dtype=int)

```

```

81     return paths, labels
82
83
84 # ----- Dataset class -----
85
86 class PathDataset(Dataset):
87     def __init__(self, paths, labels, train=True):
88         self.paths = paths
89         self.labels = labels
90
91         if train:
92             self.transform = transforms.Compose([
93                 transforms.Resize((224, 224)),
94                 transforms.RandomHorizontalFlip(),
95                 transforms.ToTensor(),
96             ])
97         else:
98             self.transform = transforms.Compose([
99                 transforms.Resize((224, 224)),
100                 transforms.ToTensor(),
101             ])
102
103     def __len__(self):
104         return len(self.paths)
105
106     def __getitem__(self, idx):
107         img_path = self.paths[idx]
108         img = Image.open(img_path).convert("RGB")
109         label = int(self.labels[idx])
110         return self.transform(img), label
111
112
113 # ----- Model definitions -----
    ↳
114
115 def freeze_all(model):
116     for p in model.parameters():
117         p.requires_grad = False
118
119
120 def unfreeze_last_k_blocks(blocks, k):
121     k = min(k, len(blocks))
122     chosen = blocks[-k:]
123     for block in chosen:
124         for p in block.parameters():
125             p.requires_grad = True
126     return chosen
127
128
129 def unfreeze_random_blocks(blocks, k):
130     k = min(k, len(blocks))
131     chosen = random.sample(blocks, k)
132     for block in chosen:
133         for p in block.parameters():
134             p.requires_grad = True
135     return chosen
136
137
138 class ResNet18_FT(nn.Module):
139     def __init__(self, num_classes, dropout=0.0):
140         super().__init__()
141         self.model = models.resnet18(weights=models.
    ↳ ResNet18_Weights.IMAGENET1K_V1)
142         in_features = self.model.fc.in_features
143         self.model.fc = nn.Sequential(
144             nn.Dropout(dropout),
145             nn.Linear(in_features, num_classes)
146         )
147         self.blocks = [
148             self.model.layer1,
149             self.model.layer2,
150             self.model.layer3,
151             self.model.layer4,
152         ]
153
154     def forward(self, x):
155         return self.model(x)
156
157
158 class MobileNetV2_FT(nn.Module):
159     def __init__(self, num_classes, dropout=0.0):
160         super().__init__()
161         self.model = models.mobilenet_v2(weights=models.
    ↳ MobileNet_V2_Weights.IMAGENET1K_V1)

```

```

162         in_features = self.model.classifier[1].
163         ↪ in_features
164         self.model.classifier = nn.Sequential(
165             nn.Dropout(dropout),
166             nn.Linear(in_features, num_classes)
167         )
168         self.blocks = [
169             self.model.features[0:2],
170             self.model.features[2:4],
171             self.model.features[4:7],
172             self.model.features[7:14],
173         ]
174     def forward(self, x):
175         return self.model(x)
176
177 # ----- Hyperparams & training
178 ↪ -----
179
180 hyperparams = {
181     "lr": [1e-2, 1e-3, 5e-4, 1e-4, 1e-5],
182     "optimizer": ["adam", "sgd", "rmsprop"],
183     "dropout": [0.1, 0.2, 0.3],
184     "k_blocks": [1, 2, 3, 4],
185     "weight_decay": [0.0, 1e-4],
186 }
187
188 def make_optimizer(optimizer_name, params, lr,
189     ↪ weight_decay):
190     name = optimizer_name.lower()
191     if name == "adam":
192         return torch.optim.Adam(params, lr=lr,
193             ↪ weight_decay=weight_decay)
194     elif name == "sgd":
195         return torch.optim.SGD(params, lr=lr, momentum=0.
196             ↪ 9, weight_decay=weight_decay)
197     elif name == "rmsprop":
198         return torch.optim.RMSprop(params, lr=lr,
199             ↪ weight_decay=weight_decay)
200     else:
201         raise ValueError(f"Unknown optimizer: {
202             ↪ optimizer_name}")
203
204 def run_trial(
205     model_class,
206     train_loader,
207     val_loader,
208     num_classes,
209     lr,
210     optimizer_name,
211     dropout,
212     k_blocks,
213     weight_decay,
214     mode="random"
215 ):
216     model = model_class(num_classes=num_classes, dropout
217         ↪ =dropout).to(DEVICE)
218     freeze_all(model)
219
220     if mode == "random":
221         chosen_blocks = unfreeze_random_blocks(model.
222             ↪ blocks, k_blocks)
223     elif mode == "last":
224         chosen_blocks = unfreeze_last_k_blocks(model.
225             ↪ blocks, k_blocks)
226     else:
227         raise ValueError("mode must be 'random' or 'last'
228             ↪ ")
229
230     optimizer = make_optimizer(
231         optimizer_name,
232         filter(lambda p: p.requires_grad, model.
233             ↪ parameters()),
234         lr=lr,
235         weight_decay=weight_decay
236     )
237     criterion = nn.CrossEntropyLoss()
238
239     EPOCHS = 5
240     for epoch in range(EPOCHS):
241         model.train()
242         for x, y in train_loader:

```

```

235         x, y = x.to(DEVICE), y.to(DEVICE)
236         optimizer.zero_grad()
237         out = model(x)
238         loss = criterion(out, y)
239         loss.backward()
240         optimizer.step()
241
242 # evaluate
243 model.eval()
244 all_preds = []
245 all_labels = []
246 with torch.no_grad():
247     for x, y in val_loader:
248         x, y = x.to(DEVICE), y.to(DEVICE)
249         out = model(x)
250         pred = out.argmax(dim=1)
251         all_preds.extend(pred.cpu().numpy())
252         all_labels.extend(y.cpu().numpy())
253
254 all_preds = np.array(all_preds)
255 all_labels = np.array(all_labels)
256
257 acc = (all_preds == all_labels).mean()
258 f1_macro = f1_score(all_labels, all_preds, average="
259     ↪ macro")
260 f1_weighted = f1_score(all_labels, all_preds,
261     ↪ average="weighted")
262
263 return acc, f1_macro, f1_weighted, chosen_blocks
264
265 def grid_search(model_class, model_name, train_loader,
266     ↪ val_loader, num_classes, mode, out_csv):
267     all_params = list(itertools.product(
268         hyperparams["lr"],
269         hyperparams["optimizer"],
270         hyperparams["dropout"],
271         hyperparams["k_blocks"],
272         hyperparams["weight_decay"],
273     ))
274
275 rows = []
276 for lr, opt, dr, k, wd in all_params:
277     print(f"\n[{model_name}][{mode}] lr={lr}, opt={
278         ↪ opt}, dr={dr}, k={k}, wd={wd}")
279     acc, f1_macro, f1_weighted, chosen_blocks =
280     ↪ run_trial(
281         model_class,
282         train_loader,
283         val_loader,
284         num_classes,
285         lr,
286         opt,
287         dr,
288         k,
289         wd,
290         mode=mode
291     )
292     row = {
293         "lr": lr,
294         "optimizer": opt,
295         "dropout": dr,
296         "k_blocks": k,
297         "weight_decay": wd,
298         "val_acc": acc,
299         "f1_macro": f1_macro,
300         "f1_weighted": f1_weighted,
301     }
302     rows.append(row)
303     print("Val acc:", acc, "F1 macro:", f1_macro)
304
305 df = pd.DataFrame(rows)
306 df.to_csv(out_csv, index=False)
307 df_sorted = df.sort_values("val_acc", ascending=
308     ↪ False)
309 best = df_sorted.iloc[0].to_dict()
310 return best
311
312 def main():
313     # --- build or load splits ---
314     if not (SPLIT_DIR / "train_paths.txt").exists():
315         print("Creating dataset splits...")
316         paths, labels, classes, _ =
317         ↪ build_paths_and_labels()

```

```

313     num_classes = len(classes)
314
315     # stratified splits
316     sss1 = StratifiedShuffleSplit(n_splits=1,
317     ↪ test_size=0.30, random_state=42)
318     train_idx, temp_idx = next(sss1.split(paths,
319     ↪ labels))
320
321     val_test_labels = labels[temp_idx]
322     sss2 = StratifiedShuffleSplit(n_splits=1,
323     ↪ test_size=0.50, random_state=42)
324     val_idx, test_idx = next(sss2.split(temp_idx,
325     ↪ val_test_labels))
326
327     train_paths = paths[train_idx]
328     val_paths = paths[temp_idx[val_idx]]
329     test_paths = paths[temp_idx[test_idx]]
330
331     train_labels = labels[train_idx]
332     val_labels = labels[temp_idx[val_idx]]
333     test_labels = labels[temp_idx[test_idx]]
334
335     # save full splits
336     save_split("train", train_paths, train_labels)
337     save_split("val", val_paths, val_labels)
338     save_split("test", test_paths, test_labels)
339
340     # create subsets for grid search
341     train_paths_sub, train_labels_sub = make_subset(
342     ↪ train_paths, train_labels, fraction=0.5)
343     val_paths_sub, val_labels_sub = make_subset(
344     ↪ val_paths, val_labels, fraction=0.5)
345
346     save_split("train_sub", train_paths_sub,
347     ↪ train_labels_sub)
348     save_split("val_sub", val_paths_sub,
349     ↪ val_labels_sub)
350
351     # --- load subset splits for grid search ---
352     train_paths_sub, train_labels_sub = load_split("
353     ↪ train_sub")
354     val_paths_sub, val_labels_sub = load_split("
355     ↪ val_sub")
356
357     num_classes = len(np.unique(train_labels_sub))
358
359     train_ds = PathDataset(train_paths_sub,
360     ↪ train_labels_sub, train=True)
361     val_ds = PathDataset(val_paths_sub,
362     ↪ val_labels_sub, train=False)
363
364     train_loader = DataLoader(train_ds, batch_size=64,
365     ↪ shuffle=True, num_workers=4)
366     val_loader = DataLoader(val_ds, batch_size=64,
367     ↪ shuffle=False, num_workers=4)
368
369     grid_dir = RESULTS_ROOT / "grid"
370     grid_dir.mkdir(exist_ok=True)
371
372     # ----- ResNet grid search -----
373     best_resnet_random = grid_search(
374     ↪ ResNet18_FT, "ResNet18",
375     ↪ train_loader, val_loader, num_classes,
376     ↪ mode="random",
377     ↪ out_csv=grid_dir / "resnet_random_results.csv"
378     )
379     best_resnet_last = grid_search(
380     ↪ ResNet18_FT, "ResNet18",
381     ↪ train_loader, val_loader, num_classes,
382     ↪ mode="last",
383     ↪ out_csv=grid_dir / "resnet_last_results.csv"
384     )
385
386     # ----- MobileNet grid search -----
387     best_mobile_random = grid_search(
388     ↪ MobileNetV2_FT, "MobileNetV2",
389     ↪ train_loader, val_loader, num_classes,
390     ↪ mode="random",
391     ↪ out_csv=grid_dir / "mobilenet_random_results.csv"
392     ↪ "
393     )
394     best_mobile_last = grid_search(
395     ↪ MobileNetV2_FT, "MobileNetV2",
396     ↪ train_loader, val_loader, num_classes,
397     ↪ mode="last",

```

```

383     out_csv=grid_dir / "mobilenet_last_results.csv"
384     )
385
386     # save best configs
387     best_config = {
388     ↪ "num_classes": int(num_classes),
389     ↪ "resnet_random": best_resnet_random,
390     ↪ "resnet_last": best_resnet_last,
391     ↪ "mobilenet_random": best_mobile_random,
392     ↪ "mobilenet_last": best_mobile_last,
393     }
394
395     with open(grid_dir / "best_configs.json", "w") as f:
396     ↪ json.dump(best_config, f, indent=4)
397
398     print("\nSaved best configs to", grid_dir / "
399     ↪ best_configs.json")
400
401     if __name__ == "__main__":
402     ↪ main()

```

Code 2. Finetuning Code

D. Full Training using best hyperparameters

```

1  import json
2  from pathlib import Path
3
4  import numpy as np
5  from sklearn.metrics import f1_score, confusion_matrix,
6  ↪ classification_report
7
8  import torch
9  import torch.nn as nn
10 from torch.utils.data import Dataset, DataLoader
11 from torchvision import transforms, models
12 from PIL import Image
13
14
15 PROJECT_ROOT = Path("/scratch/c00714ij/project")
16 DATASET_ROOT = PROJECT_ROOT / "datasets" / "MyDataset"
17
18 RESULTS_ROOT = PROJECT_ROOT / "results"
19 SPLIT_DIR = PROJECT_ROOT / "splits"
20 MODELS_DIR = PROJECT_ROOT / "models"
21
22 RESULTS_ROOT.mkdir(parents=True, exist_ok=True)
23 MODELS_DIR.mkdir(parents=True, exist_ok=True)
24
25 DEVICE = torch.device("cuda" if torch.cuda.is_available()
26 ↪ else "cpu")
27 print("Using device:", DEVICE)
28
29 class PathDataset(Dataset):
30     def __init__(self, paths, labels, train=True):
31     ↪ self.paths = paths
32     ↪ self.labels = labels
33
34     if train:
35     ↪ self.transform = transforms.Compose([
36     ↪     transforms.Resize((224, 224)),
37     ↪     transforms.RandomHorizontalFlip(),
38     ↪     transforms.ToTensor(),
39     ])
40     else:
41     ↪ self.transform = transforms.Compose([
42     ↪     transforms.Resize((224, 224)),
43     ↪     transforms.ToTensor(),
44     ])
45
46     def __len__(self):
47     ↪ return len(self.paths)
48
49     def __getitem__(self, idx):
50     ↪ img_path = self.paths[idx]
51     ↪ img = Image.open(img_path).convert("RGB")
52     ↪ label = int(self.labels[idx])
53     ↪ return self.transform(img), label
54
55

```



```

56 def load_split(name):
57     paths = np.loadtxt(SPLIT_DIR / f"{name}_paths.txt",
58         ↪ dtype=str)
59     labels = np.loadtxt(SPLIT_DIR / f"{name}_labels.txt",
60         ↪ dtype=int)
61     return np.array(paths), np.array(labels)
62
63 def freeze_all(model):
64     for p in model.parameters():
65         p.requires_grad = False
66
67 def unfreeze_last_k_blocks(blocks, k):
68     k = min(k, len(blocks))
69     chosen = blocks[-k:]
70     for block in chosen:
71         for p in block.parameters():
72             p.requires_grad = True
73     return chosen
74
75 def unfreeze_random_blocks(blocks, k):
76     import random
77     k = min(k, len(blocks))
78     chosen = random.sample(blocks, k)
79     for block in chosen:
80         for p in block.parameters():
81             p.requires_grad = True
82     return chosen
83
84 class ResNet18_FT(nn.Module):
85     def __init__(self, num_classes, dropout=0.0):
86         super().__init__()
87         self.model = models.resnet18(weights=models.
88             ↪ ResNet18_Weights.IMAGENET1K_V1)
89         in_features = self.model.fc.in_features
90         self.model.fc = nn.Sequential(
91             nn.Dropout(dropout),
92             nn.Linear(in_features, num_classes)
93         )
94         self.blocks = [
95             self.model.layer1,
96             self.model.layer2,
97             self.model.layer3,
98             self.model.layer4,
99         ]
100
101     def forward(self, x):
102         return self.model(x)
103
104 class MobileNetV2_FT(nn.Module):
105     def __init__(self, num_classes, dropout=0.0):
106         super().__init__()
107         self.model = models.mobilenet_v2(weights=models.
108             ↪ MobileNet_V2_Weights.IMAGENET1K_V1)
109         in_features = self.model.classifier[1].
110             ↪ in_features
111         self.model.classifier = nn.Sequential(
112             nn.Dropout(dropout),
113             nn.Linear(in_features, num_classes)
114         )
115         self.blocks = [
116             self.model.features[0:2],
117             self.model.features[2:4],
118             self.model.features[4:7],
119             self.model.features[7:14],
120         ]
121
122     def forward(self, x):
123         return self.model(x)
124
125 def make_optimizer(optimizer_name, params, lr,
126     ↪ weight_decay):
127     name = optimizer_name.lower()
128     if name == "adam":
129         return torch.optim.Adam(params, lr=lr,
130             ↪ weight_decay=weight_decay)
131     elif name == "sgd":
132         return torch.optim.SGD(params, lr=lr, momentum=0.
133             ↪ 9, weight_decay=weight_decay)
134
135     elif name == "rmsprop":
136         return torch.optim.RMSprop(params, lr=lr,
137             ↪ weight_decay=weight_decay)
138     else:
139         raise ValueError(f"Unknown optimizer: {
140             ↪ optimizer_name}")
141
142 def evaluate_model(model, loader, device, num_classes):
143     model.eval()
144     all_preds = []
145     all_labels = []
146
147     with torch.no_grad():
148         for x, y in loader:
149             x, y = x.to(device), y.to(device)
150             out = model(x)
151             pred = out.argmax(dim=1)
152             all_preds.extend(pred.cpu().numpy())
153             all_labels.extend(y.cpu().numpy())
154
155     all_preds = np.array(all_preds)
156     all_labels = np.array(all_labels)
157
158     acc = (all_preds == all_labels).mean()
159     f1_macro = f1_score(all_labels, all_preds, average="
160         ↪ macro")
161     f1_weighted = f1_score(all_labels, all_preds,
162         ↪ average="weighted")
163     f1_per_class = f1_score(all_labels, all_preds,
164         ↪ average=None, labels=np.arange(num_classes))
165     cm = confusion_matrix(all_labels, all_preds, labels=
166         ↪ np.arange(num_classes))
167     report = classification_report(all_labels, all_preds,
168         ↪ labels=np.arange(num_classes))
169
170     return {
171         "accuracy": float(acc),
172         "f1_macro": float(f1_macro),
173         "f1_weighted": float(f1_weighted),
174         "f1_per_class": f1_per_class.tolist(),
175         "confusion_matrix": cm.tolist(),
176         "report": report,
177     }
178
179 def full_train(model_class, config, mode, train_loader,
180     ↪ val_loader, num_classes, epochs, save_path):
181     lr = config["lr"]
182     optimizer_name = config["optimizer"]
183     dropout = config["dropout"]
184     k_blocks = int(config["k_blocks"])
185     weight_decay = config["weight_decay"]
186
187     model = model_class(num_classes=num_classes, dropout
188         ↪ =dropout).to(DEVICE)
189     freeze_all(model)
190
191     if mode == "random":
192         unfreeze_random_blocks(model.blocks, k_blocks)
193     else:
194         unfreeze_last_k_blocks(model.blocks, k_blocks)
195
196     optimizer = make_optimizer(
197         optimizer_name,
198         filter(lambda p: p.requires_grad, model.
199             ↪ parameters()),
200         lr=lr,
201         weight_decay=weight_decay
202     )
203     criterion = nn.CrossEntropyLoss()
204
205     best_acc = 0.0
206     for epoch in range(epochs):
207         model.train()
208         for x, y in train_loader:
209             x, y = x.to(DEVICE), y.to(DEVICE)
210             optimizer.zero_grad()
211             out = model(x)
212             loss = criterion(out, y)
213             loss.backward()
214             optimizer.step()
215
216         # val
217         model.eval()

```

```

208     correct = 0
209     total = 0
210     with torch.no_grad():
211         for x, y in val_loader:
212             x, y = x.to(DEVICE), y.to(DEVICE)
213             out = model(x)
214             pred = out.argmax(dim=1)
215             correct += (pred == y).sum().item()
216             total += y.size(0)
217
218     acc = correct / total
219     print(f"[{save_path.name}] Epoch {epoch+1}/{\
↪ epochs} Val Acc: {acc:.4f}")
220
221     if acc > best_acc:
222         best_acc = acc
223         torch.save(model.state_dict(), save_path)
224
225     print(f"Finished training {save_path.name}, best val
↪ acc = {best_acc:.4f}")
226     return best_acc
227
228 def main():
229     # load best configs from grid search
230     grid_dir = RESULTS_ROOT / "grid"
231     with open(grid_dir / "best_configs.json") as f:
232         best_cfg = json.load(f)
233
234     num_classes = int(best_cfg["num_classes"])
235
236     # load full splits
237     train_paths, train_labels = load_split("train")
238     val_paths, val_labels = load_split("val")
239     test_paths, test_labels = load_split("test")
240
241     train_ds = PathDataset(train_paths, train_labels,
↪ train=True)
242     val_ds = PathDataset(val_paths, val_labels,
↪ train=False)
243     test_ds = PathDataset(test_paths, test_labels,
↪ train=False)
244
245     train_loader = DataLoader(train_ds, batch_size=64,
↪ shuffle=True, num_workers=4)
246     val_loader = DataLoader(val_ds, batch_size=64,
↪ shuffle=False, num_workers=4)
247     test_loader = DataLoader(test_ds, batch_size=64,
↪ shuffle=False, num_workers=4)
248
249     finetune_dir = RESULTS_ROOT / "finetune"
250     finetune_dir.mkdir(exist_ok=True)
251
252     EPOCHS = 10 # you can increase later
253
254     # ----- ResNet18 -----
255     resnet_random_cfg = best_cfg["resnet_random"]
256     resnet_last_cfg = best_cfg["resnet_last"]
257
258     resnet_random_path = MODELS_DIR / "
↪ resnet_best_random.pth"
259     resnet_last_path = MODELS_DIR / "resnet_best_last.
↪.pth"
260
261     full_train(
262         ResNet18_FT,
263         resnet_random_cfg,
264         mode="random",
265         train_loader=train_loader,
266         val_loader=val_loader,
267         num_classes=num_classes,
268         epochs=EPOCHS,
269         save_path=resnet_random_path
270     )
271
272     full_train(
273         ResNet18_FT,
274         resnet_last_cfg,
275         mode="last",
276         train_loader=train_loader,
277         val_loader=val_loader,
278         num_classes=num_classes,
279         epochs=EPOCHS,
280         save_path=resnet_last_path
281     )
282
283     # ----- MobileNetV2 -----
284     mobile_random_cfg = best_cfg["mobilenet_random"]
285     mobile_last_cfg = best_cfg["mobilenet_last"]
286
287     mobile_random_path = MODELS_DIR / "
↪ mobile_best_random.pth"
288     mobile_last_path = MODELS_DIR / "mobile_best_last.
↪.pth"
289
290     full_train(
291         MobileNetV2_FT,
292         mobile_random_cfg,
293         mode="random",
294         train_loader=train_loader,
295         val_loader=val_loader,
296         num_classes=num_classes,
297         epochs=EPOCHS,
298         save_path=mobile_random_path
299     )
300
301     full_train(
302         MobileNetV2_FT,
303         mobile_last_cfg,
304         mode="last",
305         train_loader=train_loader,
306         val_loader=val_loader,
307         num_classes=num_classes,
308         epochs=EPOCHS,
309         save_path=mobile_last_path
310     )
311
312     # ----- evaluate all 4 on test set -----
313     summary = {}
314
315     # ResNet random
316     model = ResNet18_FT(num_classes, dropout=
↪ resnet_random_cfg["dropout"]).to(DEVICE)
317     model.load_state_dict(torch.load(resnet_random_path,
↪ map_location=DEVICE))
318     metrics = evaluate_model(model, test_loader, DEVICE,
↪ num_classes)
319     summary["resnet_random"] = metrics
320
321     # ResNet last
322     model = ResNet18_FT(num_classes, dropout=
↪ resnet_last_cfg["dropout"]).to(DEVICE)
323     model.load_state_dict(torch.load(resnet_last_path,
↪ map_location=DEVICE))
324     metrics = evaluate_model(model, test_loader, DEVICE,
↪ num_classes)
325     summary["resnet_last"] = metrics
326
327     # Mobile random
328     model = MobileNetV2_FT(num_classes, dropout=
↪ mobile_random_cfg["dropout"]).to(DEVICE)
329     model.load_state_dict(torch.load(mobile_random_path,
↪ map_location=DEVICE))
330     metrics = evaluate_model(model, test_loader, DEVICE,
↪ num_classes)
331     summary["mobile_random"] = metrics
332
333     # Mobile last
334     model = MobileNetV2_FT(num_classes, dropout=
↪ mobile_last_cfg["dropout"]).to(DEVICE)
335     model.load_state_dict(torch.load(mobile_last_path,
↪ map_location=DEVICE))
336     metrics = evaluate_model(model, test_loader, DEVICE,
↪ num_classes)
337     summary["mobile_last"] = metrics
338
339     with open(finetune_dir / "test_metrics.json", "w") as
↪ f:
340         json.dump(summary, f, indent=4)
341
342     print("Saved test metrics to", finetune_dir / "
↪ test_metrics.json")
343
344     if __name__ == "__main__":
345         main()
346

```

Code 3. Full training Code

E. Ensemble Classifier

```

1 import json
2 from pathlib import Path
3
4 import numpy as np
5 from sklearn.metrics import f1_score, confusion_matrix,
    ↪ classification_report
6
7 import torch
8 import torch.nn as nn
9 import torch.nn.functional as F
10 from torch.utils.data import Dataset, DataLoader
11 from torchvision import transforms, models
12 from PIL import Image
13
14 # ===== PATHS (CSF SCRATCH VERSION) =====
15 PROJECT_ROOT = Path("/scratch/c00714ij/project")
16 DATASET_ROOT = PROJECT_ROOT / "datasets" / "MyDataset"
17
18 RESULTS_ROOT = PROJECT_ROOT / "results"
19 SPLIT_DIR = PROJECT_ROOT / "splits"
20 MODELS_DIR = PROJECT_ROOT / "models"
21
22 RESULTS_ROOT.mkdir(parents=True, exist_ok=True)
23 MODELS_DIR.mkdir(parents=True, exist_ok=True)
24
25 # =====
26 DEVICE = torch.device("cuda" if torch.cuda.is_available()
    ↪ else "cpu")
27 print("Using device:", DEVICE)
28
29 # ----- Dataset -----
30 class PathDataset(Dataset):
31     def __init__(self, paths, labels, train=True):
32         self.paths = paths
33         self.labels = labels
34
35         if train:
36             self.transform = transforms.Compose([
37                 transforms.Resize((224, 224)),
38                 transforms.RandomHorizontalFlip(),
39                 transforms.ToTensor(),
40             ])
41         else:
42             self.transform = transforms.Compose([
43                 transforms.Resize((224, 224)),
44                 transforms.ToTensor(),
45             ])
46
47     def __len__(self):
48         return len(self.paths)
49
50     def __getitem__(self, idx):
51         img = Image.open(self.paths[idx]).convert("RGB")
52         label = int(self.labels[idx])
53         return self.transform(img), label
54
55 def load_split(name):
56     paths = np.loadtxt(SPLIT_DIR / f"{name}_paths.txt",
57         ↪ dtype=str)
58     labels = np.loadtxt(SPLIT_DIR / f"{name}_labels.txt",
59         ↪ dtype=int)
60     return paths, labels
61
62 # ----- Backbone Models -----
63 class ResNet18_FT(nn.Module):
64     def __init__(self, num_classes, dropout=0.0):
65         super().__init__()
66         self.model = models.resnet18(weights=models.
67             ↪ ResNet18_Weights.IMAGENET1K_V1)
68         in_features = self.model.fc.in_features
69         self.model.fc = nn.Sequential(nn.Dropout(dropout)
70             ↪ ,
71                                     nn.Linear(
72             ↪ in_features, num_classes))
73         self.blocks = [
74             self.model.layer1,
75             self.model.layer2,
76             self.model.layer3,
77             self.model.layer4,

```

```

77 ]
78
79 def forward(self, x):
80     return self.model(x)
81
82 class MobileNetV2_FT(nn.Module):
83     def __init__(self, num_classes, dropout=0.0):
84         super().__init__()
85         self.model = models.mobilenet_v2(weights=models.
86             ↪ MobileNet_V2_Weights.IMAGENET1K_V1)
87         in_features = self.model.classifier[1].
88             ↪ in_features
89         self.model.classifier = nn.Sequential(nn.Dropout(
90             ↪ (dropout),
91                                     nn.Linear(
92             ↪ in_features, num_classes))
93
94         self.blocks = [
95             self.model.features[0:2],
96             self.model.features[2:4],
97             self.model.features[4:7],
98             self.model.features[7:14],
99         ]
100
101 def forward(self, x):
102     return self.model(x)
103
104 # ----- Helper -----
105 def evaluate_model(model, loader, device, num_classes):
106     model.eval()
107     preds, labels = [], []
108
109     with torch.no_grad():
110         for x, y in loader:
111             x, y = x.to(device), y.to(device)
112             p = model(x).argmax(dim=1)
113             preds.extend(p.cpu().numpy())
114             labels.extend(y.cpu().numpy())
115
116     preds = np.array(preds)
117     labels = np.array(labels)
118
119     return {
120         "accuracy": float((preds == labels).mean()),
121         "f1_macro": float(f1_score(labels, preds, average
122             ↪ ="macro")),
123         "f1_weighted": float(f1_score(labels, preds,
124             ↪ average="weighted")),
125         "f1_per_class": f1_score(labels, preds, average=
126             ↪ None).tolist(),
127         "confusion_matrix": confusion_matrix(labels,
128             ↪ preds).tolist(),
129         "report": classification_report(labels, preds),
130     }
131
132 def unfreeze_last_block(blocks):
133     last_block = blocks[-1]
134     for p in last_block.parameters():
135         p.requires_grad = True
136     return last_block
137
138 # ----- Ensemble Model -----
139 class EnsembleModel(nn.Module):
140     def __init__(self, resnet_model, mobilenet_model,
141         ↪ num_classes, freeze_backbone=True):
142         super().__init__()
143
144         # Correct feature extraction
145         self.resnet_features = nn.Sequential(*list(
146             ↪ resnet_model.model.children())[:-1])
147         self.mobilenet_features = mobilenet_model.model.
148             ↪ features
149
150         # keep blocks for unfreezing
151         self.resnet_blocks = resnet_model.blocks
152         self.mobilenet_blocks = mobilenet_model.blocks
153
154         if freeze_backbone:
155             for p in self.resnet_features.parameters():
156                 p.requires_grad = False

```

```

150         for p in self.mobilenet_features.parameters():
151             ↪ :
152                 p.requires_grad = False
153
154         self.classifier = nn.Sequential(
155             nn.Linear(512 + 1280, 512),
156             nn.ReLU(),
157             nn.Dropout(0.3),
158             nn.Linear(512, num_classes)
159         )
160
161     def forward(self, x):
162         r = self.resnet_features(x)
163         r = F.adaptive_avg_pool2d(r, 1).view(r.size(0), -
164             ↪ 1)
165
166         m = self.mobilenet_features(x)
167         m = F.adaptive_avg_pool2d(m, 1).view(m.size(0), -
168             ↪ 1)
169
170         return self.classifier(torch.cat([r, m], dim=1))
171
172 # ----- Training -----
173 def train_and_eval_ensemble(
174     resnet_model, mobilenet_model,
175     train_loader, val_loader, test_loader,
176     num_classes, device, mode, epochs, save_path
177 ):
178     freeze_backbone = (mode == "classifier_only")
179
180     model = EnsembleModel(
181         resnet_model, mobilenet_model,
182         num_classes, freeze_backbone
183     ).to(device)
184
185     if mode == "last_block":
186         unfreeze_last_block(model.resnet_blocks)
187         unfreeze_last_block(model.mobilenet_blocks)
188
189     opt = torch.optim.Adam(filter(lambda p: p.
190         ↪ requires_grad, model.parameters()),
191         lr=1e-3, weight_decay=1e-4)
192     crit = nn.CrossEntropyLoss()
193
194     best = 0.0
195     for epoch in range(epochs):
196         model.train()
197         for x, y in train_loader:
198             x, y = x.to(device), y.to(device)
199             opt.zero_grad()
200             loss = crit(model(x), y)
201             loss.backward()
202             opt.step()
203
204         # validation
205         model.eval()
206         correct = total = 0
207         with torch.no_grad():
208             for x, y in val_loader:
209                 x, y = x.to(device), y.to(device)
210                 pred = model(x).argmax(dim=1)
211                 correct += (pred == y).sum().item()
212                 total += y.size(0)
213
214         acc = correct / total
215         print(f"[{mode}] Epoch {epoch+1}/{epochs} Val Acc
216             ↪ = {acc:.4f}")
217
218         if acc > best:
219             best = acc
220             torch.save(model.state_dict(), save_path)
221
222     # load best + test
223     model.load_state_dict(torch.load(save_path,
224         ↪ map_location=device))
225     metrics = evaluate_model(model, test_loader, device,
226         ↪ num_classes)
227     return best, metrics
228
229 # ----- Main -----
230 def main():
231     # load best configs
232
233     with open(RESULTS_ROOT / "grid" / "best_configs.json"
234         ↪ ) as f:
235         cfg = json.load(f)
236
237     num_classes = int(cfg["num_classes"])
238
239     # load splits
240     train_paths, train_labels = load_split("train")
241     val_paths, val_labels = load_split("val")
242     test_paths, test_labels = load_split("test")
243
244     train_loader = DataLoader(PathDataset(train_paths,
245         ↪ train_labels, train=True),
246         batch_size=64, shuffle=
247         ↪ True, num_workers=4)
248     val_loader = DataLoader(PathDataset(val_paths,
249         ↪ val_labels, train=False),
250         batch_size=64, shuffle=False,
251         ↪ num_workers=4)
252     test_loader = DataLoader(PathDataset(test_paths,
253         ↪ test_labels, train=False),
254         batch_size=64, shuffle=
255         ↪ False, num_workers=4)
256
257     # Load fine-tuned backbones
258     resnet_random = ResNet18_FT(num_classes, dropout=cfg
259         ↪ ["resnet_random"]["dropout"]).to(DEVICE)
260     resnet_random.load_state_dict(torch.load(MODELS_DIR /
261         ↪ "resnet_best_random.pth", map_location=DEVICE))
262
263     mobile_random = MobileNetV2_FT(num_classes, dropout=
264         ↪ cfg["mobilenet_random"]["dropout"]).to(DEVICE)
265     mobile_random.load_state_dict(torch.load(MODELS_DIR /
266         ↪ "mobile_best_random.pth", map_location=DEVICE))
267
268     resnet_last = ResNet18_FT(num_classes, dropout=cfg["
269         ↪ resnet_last"]["dropout"]).to(DEVICE)
270     resnet_last.load_state_dict(torch.load(MODELS_DIR /
271         ↪ "resnet_best_last.pth", map_location=DEVICE))
272
273     mobile_last = MobileNetV2_FT(num_classes, dropout=
274         ↪ cfg["mobilenet_last"]["dropout"]).to(DEVICE)
275     mobile_last.load_state_dict(torch.load(MODELS_DIR /
276         ↪ "mobile_best_last.pth", map_location=DEVICE))
277
278     ens_dir = RESULTS_ROOT / "ensemble"
279     ens_dir.mkdir(exist_ok=True)
280
281     summary = {}
282
283     # random-trained backbones
284     for mode in ["classifier_only", "last_block"]:
285         best, metrics = train_and_eval_ensemble(
286             resnet_random, mobile_random,
287             train_loader, val_loader, test_loader,
288             num_classes, DEVICE, mode,
289             epochs=5,
290             save_path=ens_dir / f"ensemble_random_{mode}.
291             ↪ .pth"
292         )
293         summary[f"ensemble_random_{mode}"] = {"
294             ↪ best_val_acc": best, "test_metrics": metrics}
295
296     # last-block trained backbones
297     for mode in ["classifier_only", "last_block"]:
298         best, metrics = train_and_eval_ensemble(
299             resnet_last, mobile_last,
300             train_loader, val_loader, test_loader,
301             num_classes, DEVICE, mode,
302             epochs=5,
303             save_path=ens_dir / f"ensemble_last_{mode}.
304             ↪ .pth"
305         )
306         summary[f"ensemble_last_{mode}"] = {"
307             ↪ best_val_acc": best, "test_metrics": metrics}
308
309     with open(ens_dir / "ensemble_metrics.json", "w") as
310         ↪ f:
311         json.dump(summary, f, indent=4)
312
313     print("Saved:", ens_dir / "ensemble_metrics.json")
314
315 if __name__ == "__main__":
316     main()

```

Code 4. Ensemble Classifier Code